

# Privacy-Preserving Machine Learning on Web Browsing for Public Opinion

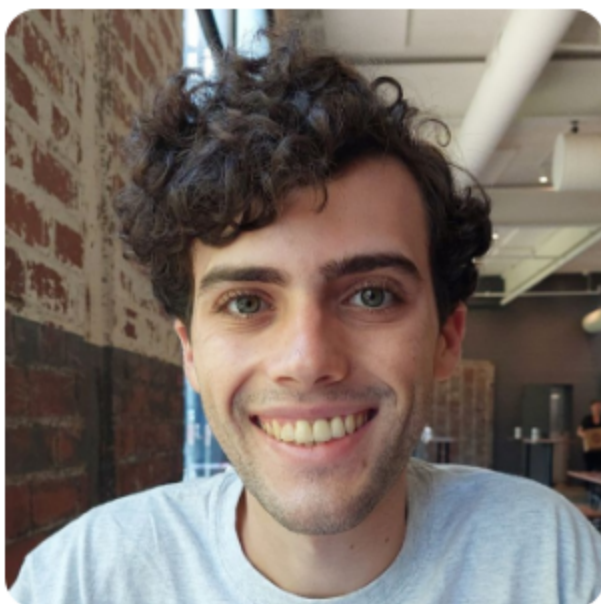
**Sam Buxbaum**

Lucas M. Tassis, Lucas Boschelli, Giovanni Comarela, Mayank Varia, Mark Crovella, Dino P. Christenson



CSCML 25

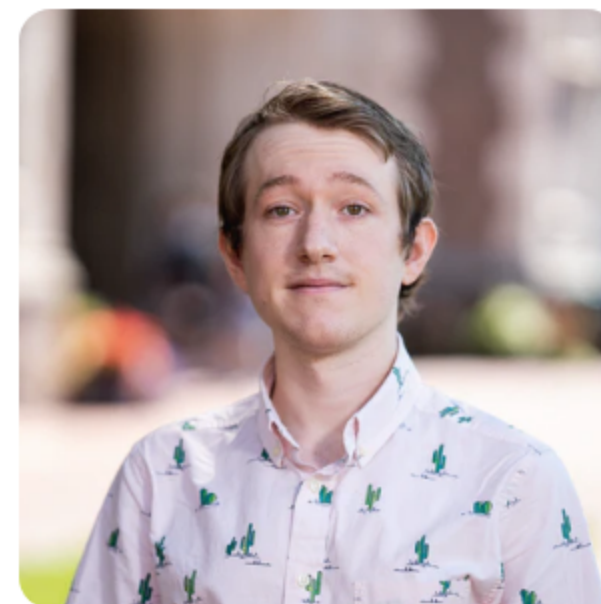
December 4, 2025



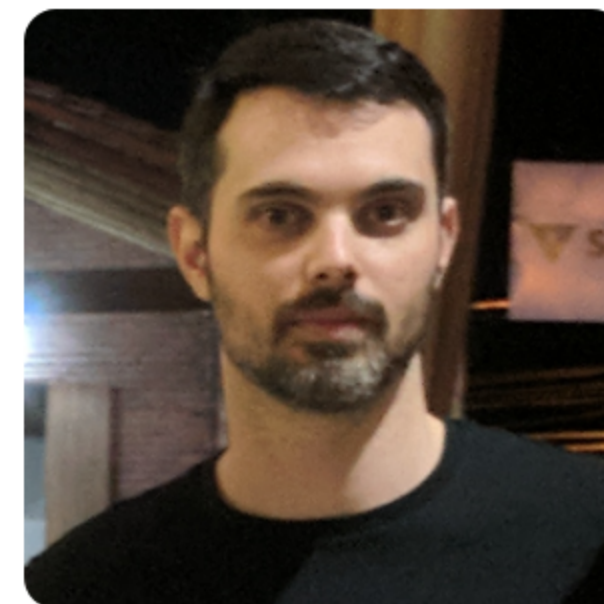
Sam Buxbaum



Lucas M. Tassis



Lucas Boschelli



Giovanni Comarela



Mayank Varia



Mark Crovella



Dino P. Christenson

# Traditional Political Polling

# Traditional Political Polling

- Data collection takes time

# Traditional Political Polling

- Data collection takes time
- Data collection is human-intensive



# Traditional Political Polling

- Data collection takes time
- Data collection is human-intensive
- Poor geographic and temporal coverage

# Traditional Political Polling

- Data collection takes time
- Data collection is human-intensive
- Poor geographic and temporal coverage

## West Virginia 2024 Presidential Election Polls



### Harris vs. Trump

| Source           | Date      | Sample            | Harris | Trump | Other |
|------------------|-----------|-------------------|--------|-------|-------|
| Research America | 8/30/2024 | 400 LV $\pm$ 4.9% | 34%    | 61%   | 5%    |

# Traditional Political Polling

- Data collection takes time
- Data collection is human-intensive
- Poor geographic and temporal coverage

## West Virginia 2024 Presidential Election Polls



Harris vs. Trump

| Source           | Date      | Sample            | Harris | Trump | Other |
|------------------|-----------|-------------------|--------|-------|-------|
| Research America | 8/30/2024 | 400 LV $\pm$ 4.9% | 34%    | 61%   | 5%    |

## Michigan 2024 Presidential Election Polls



Instantly compare a poll to prior one by same pollster

Harris vs. Trump

| Source                   | Date       | Sample              | Harris | Trump | Other |
|--------------------------|------------|---------------------|--------|-------|-------|
| Average of 23 Polls†     |            |                     | 48.6%  | 46.8% | -     |
| FAU / Mainstreet         | 11/04/2024 | 713 LV              | 49%    | 47%   | 4%    |
| Emerson College          | 11/04/2024 | 790 LV $\pm$ 3.4%   | 50%    | 48%   | 2%    |
| Research Co.             | 11/04/2024 | 450 LV $\pm$ 4.6%   | 49%    | 47%   | 4%    |
| InsiderAdvantage         | 11/03/2024 | 800 LV $\pm$ 3.7%   | 47%    | 47%   | 6%    |
| Trafalgar Group          | 11/03/2024 | 1,079 LV $\pm$ 2.9% | 47%    | 48%   | 5%    |
| MIRS / Mich. News Source | 11/03/2024 | 585 LV $\pm$ 4%     | 50%    | 48%   | 2%    |
| NY Times / Siena College | 11/03/2024 | 998 LV $\pm$ 3.7%   | 47%    | 47%   | 6%    |
| Morning Consult          | 11/03/2024 | 1,108 LV $\pm$ 3%   | 49%    | 48%   | 3%    |
| AtlasIntel               | 11/02/2024 | 1,198 LV $\pm$ 3%   | 48%    | 50%   | 2%    |
| Redfield & Wilton        | 11/01/2024 | 1,731 LV $\pm$ 2.2% | 47%    | 47%   | 6%    |
| The Times (UK) / YouGov  | 11/01/2024 | 942 LV $\pm$ 3.9%   | 48%    | 45%   | 7%    |
| EPIC-MRA                 | 11/01/2024 | 600 LV $\pm$ 4%     | 48%    | 45%   | 7%    |
| Marist Poll              | 11/01/2024 | 1,214 LV $\pm$ 3.5% | 51%    | 48%   | 1%    |
| AtlasIntel               | 10/31/2024 | 1,136 LV $\pm$ 3%   | 49%    | 49%   | 2%    |
| Echelon Insights         | 10/31/2024 | 600 LV $\pm$ 4.4%   | 48%    | 48%   | 4%    |
| MIRS / Mich. News Source | 10/31/2024 | 1,117 LV $\pm$ 2.5% | 47%    | 49%   | 4%    |
| UMass Lowell             | 10/31/2024 | 600 LV $\pm$ 4.5%   | 49%    | 45%   | 6%    |
| Washington Post          | 10/31/2024 | 1,003 LV $\pm$ 3.7% | 47%    | 46%   | 7%    |
| Fox News                 | 10/30/2024 | 988 LV $\pm$ 3%     | 49%    | 49%   | 2%    |
| CNN                      | 10/30/2024 | 726 LV $\pm$ 4.7%   | 48%    | 43%   | 9%    |
| Suffolk University       | 10/30/2024 | 500 LV $\pm$ 4.4%   | 47%    | 47%   | 6%    |



# Web Browsing for Political Polling

# Web Browsing for Political Polling

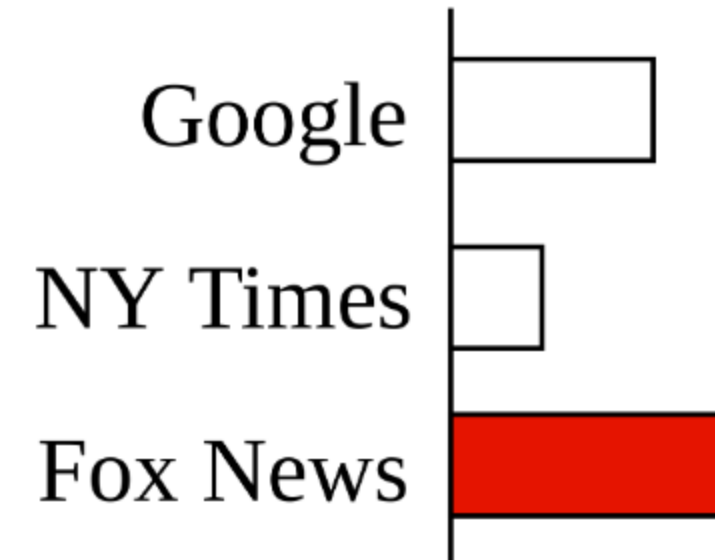
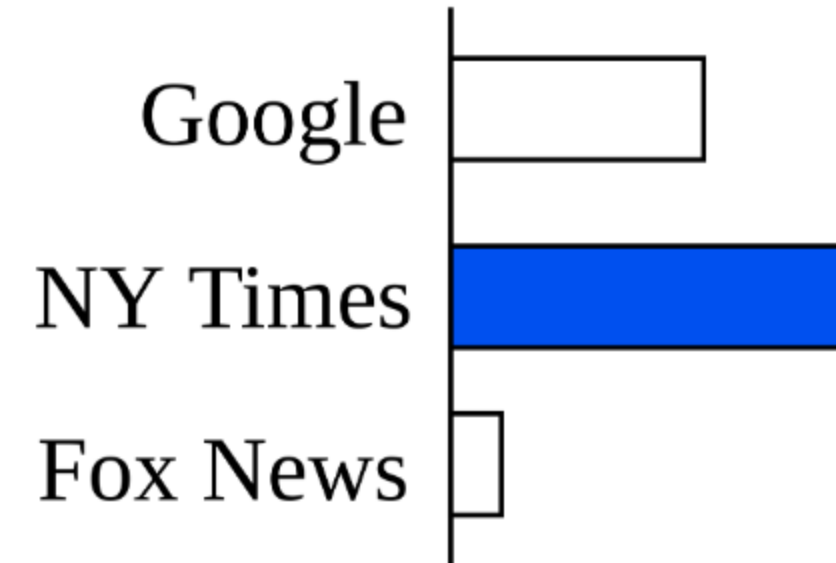
- Can website visits predict political leanings?

# Web Browsing for Political Polling

- Can website visits predict political leanings?
- Example - news websites

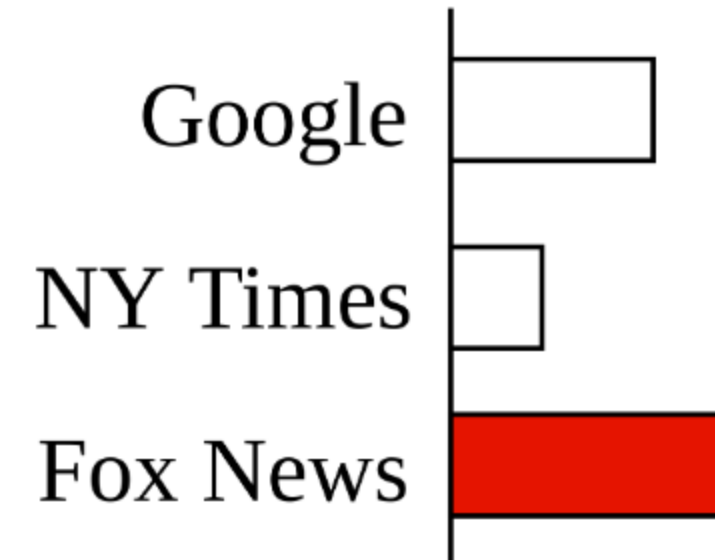
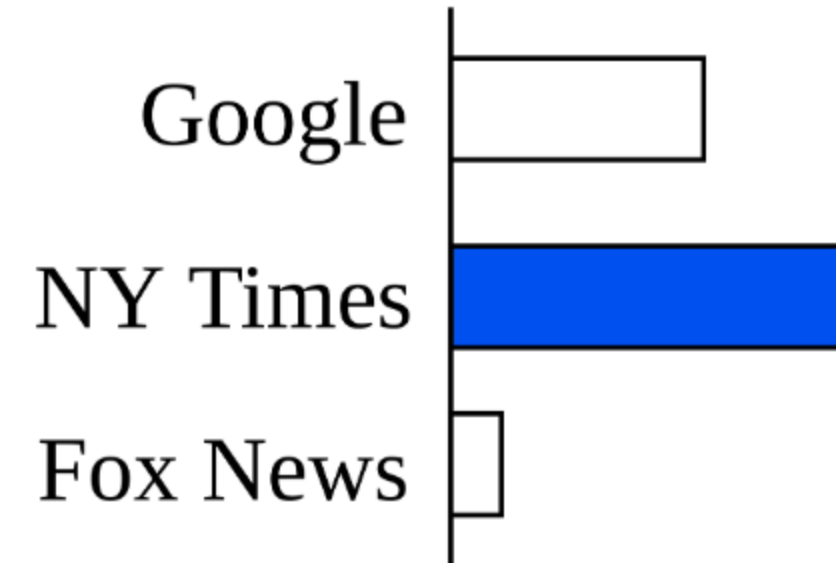
# Web Browsing for Political Polling

- Can website visits predict political leanings?
- Example - news websites



# Web Browsing for Political Polling

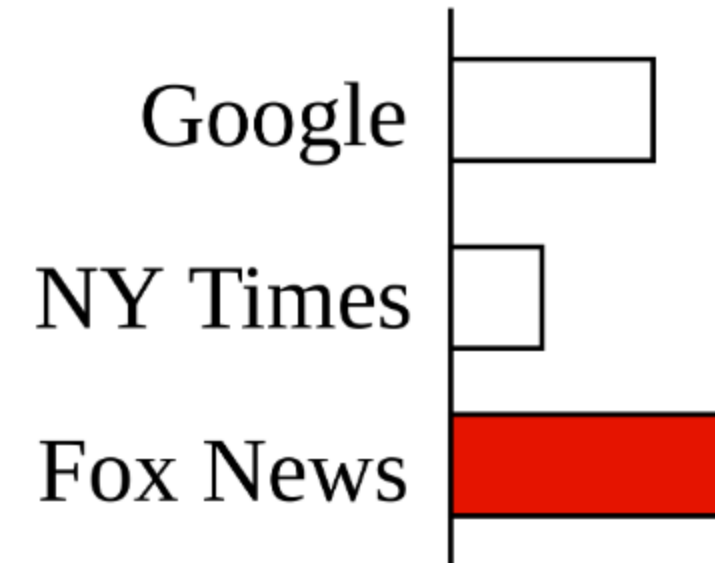
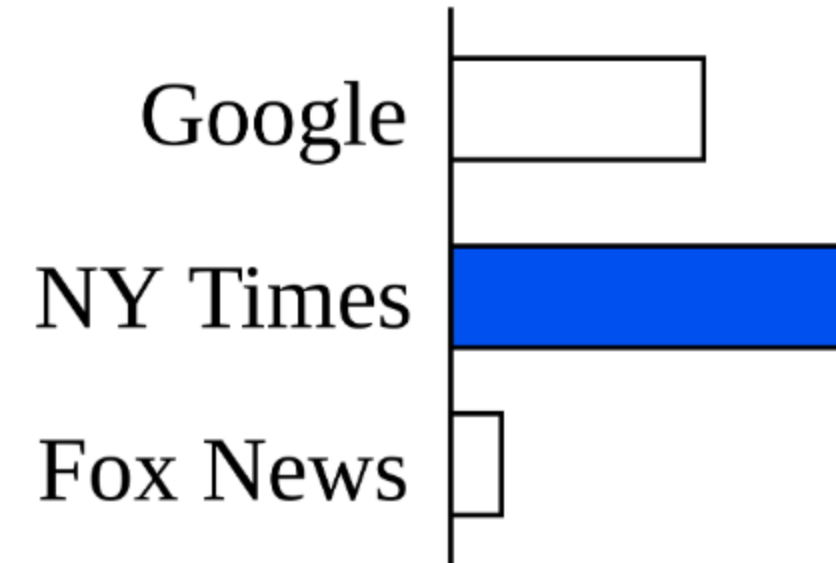
- Can website visits predict political leanings?
- Example - news websites
- More data





# Web Browsing for Political Polling

- Can website visits predict political leanings?
- Example - news websites
- More data
- Fully automated



# Prior Work

# Prior Work

- Web browsing behavior can predict voting results



# Prior Work

- Web browsing behavior can predict voting results
- Quantifying the 'Comey letter' (Comarela et al.)

# Prior Work

- Web browsing behavior can predict voting results
- Quantifying the 'Comey letter' (Comarela et al.)

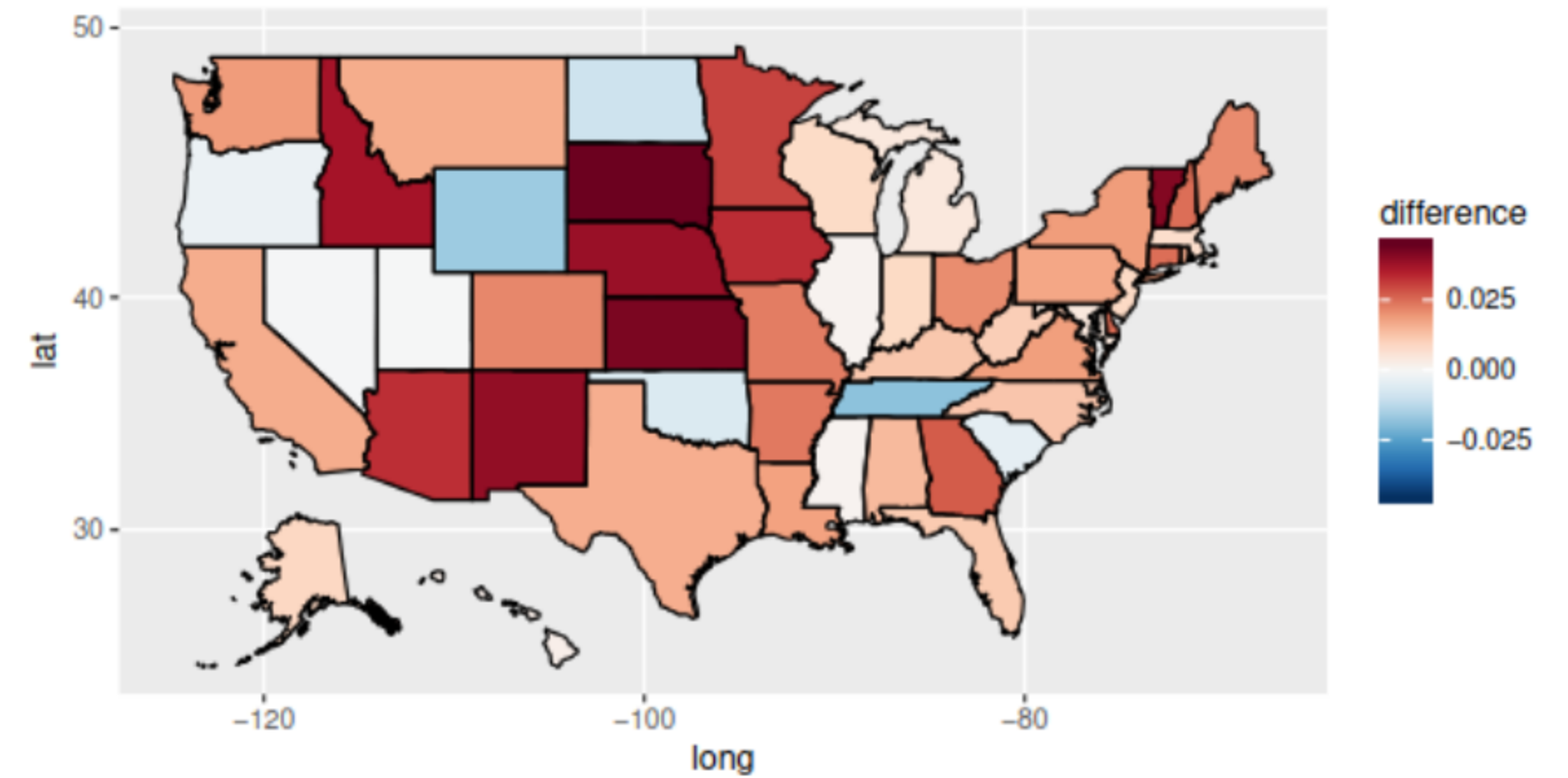


Figure 8: Impact of the 'Comey letter' at the state level.

# Prior Work

- Web browsing behavior can predict voting results
- Quantifying the 'Corney letter' (Comarella et al.)

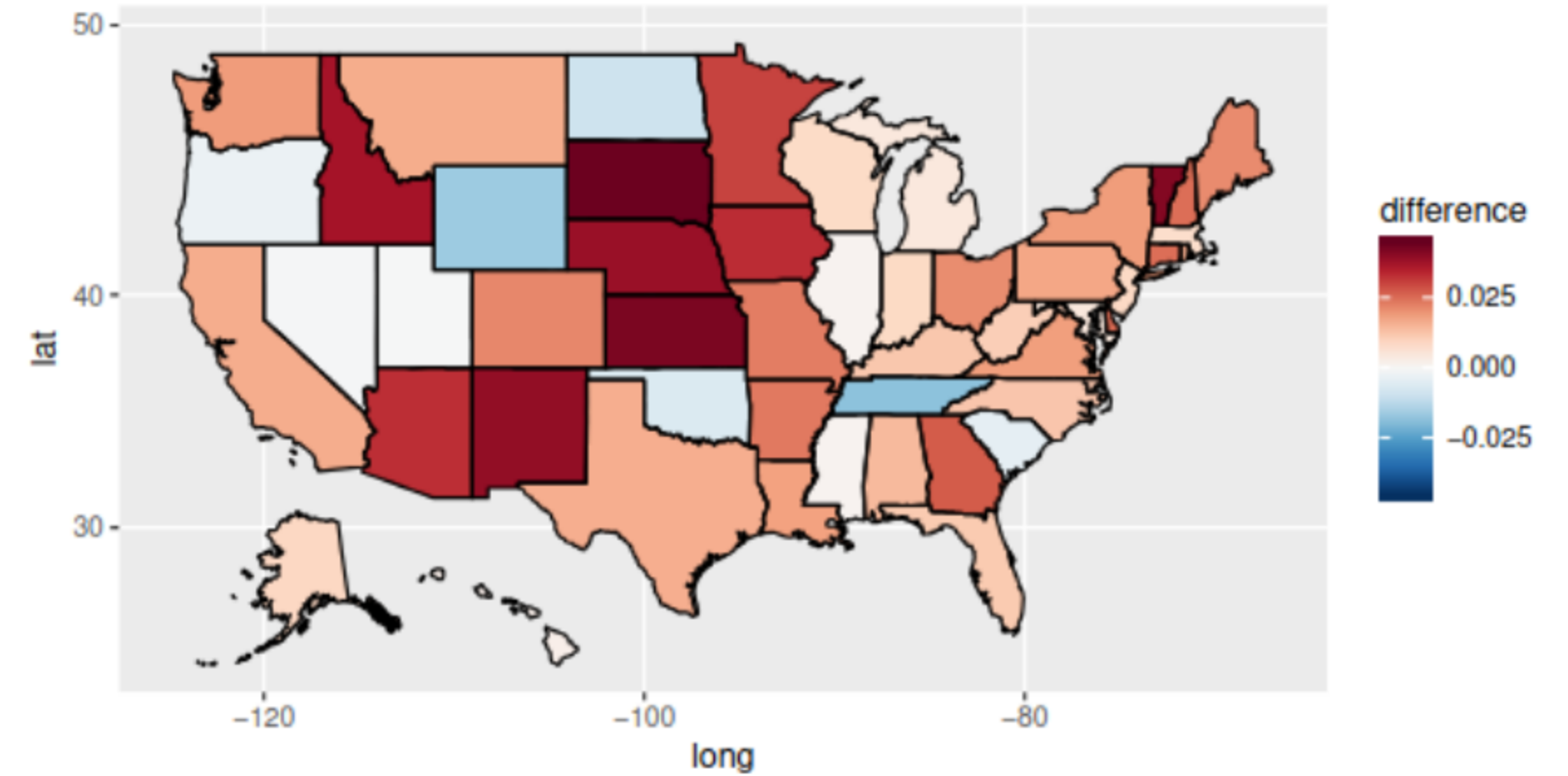
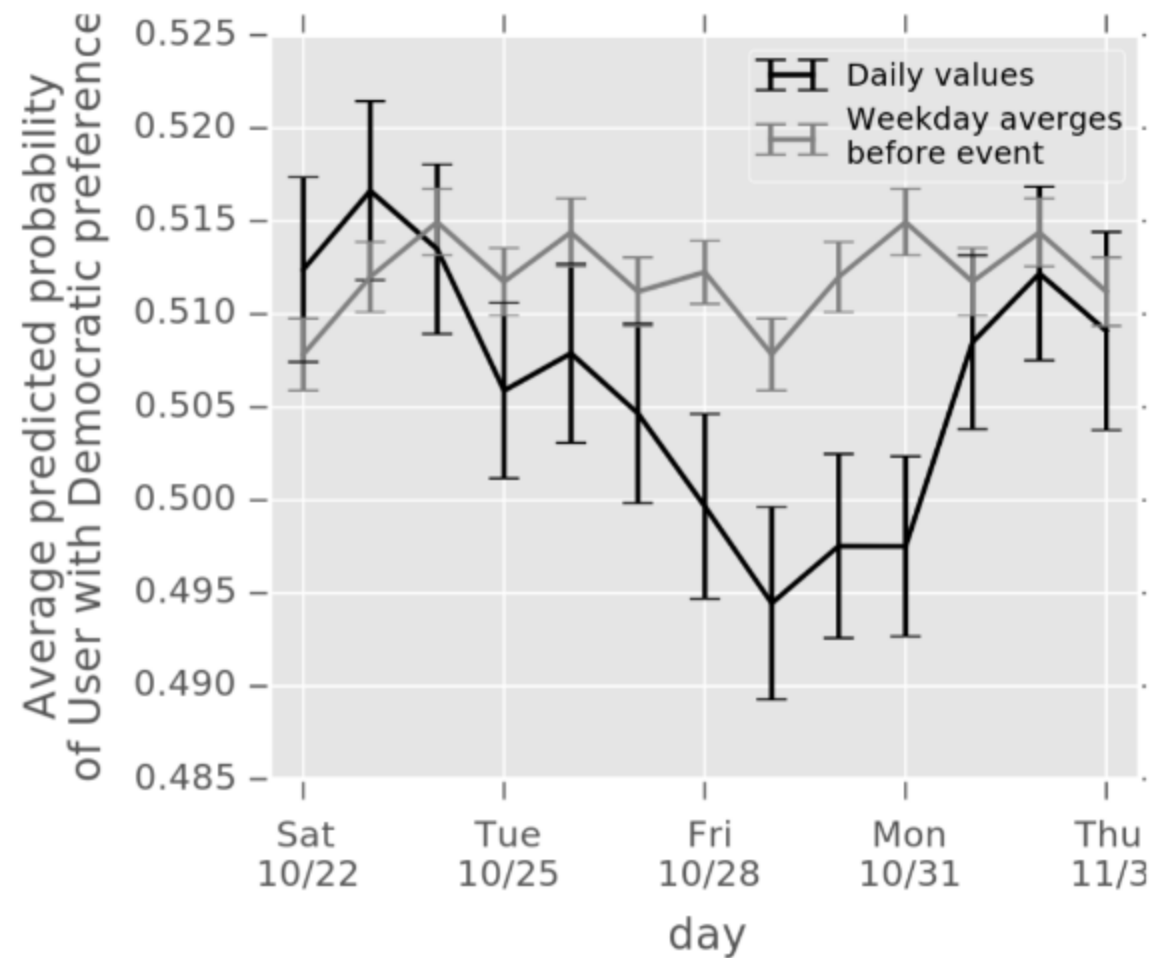
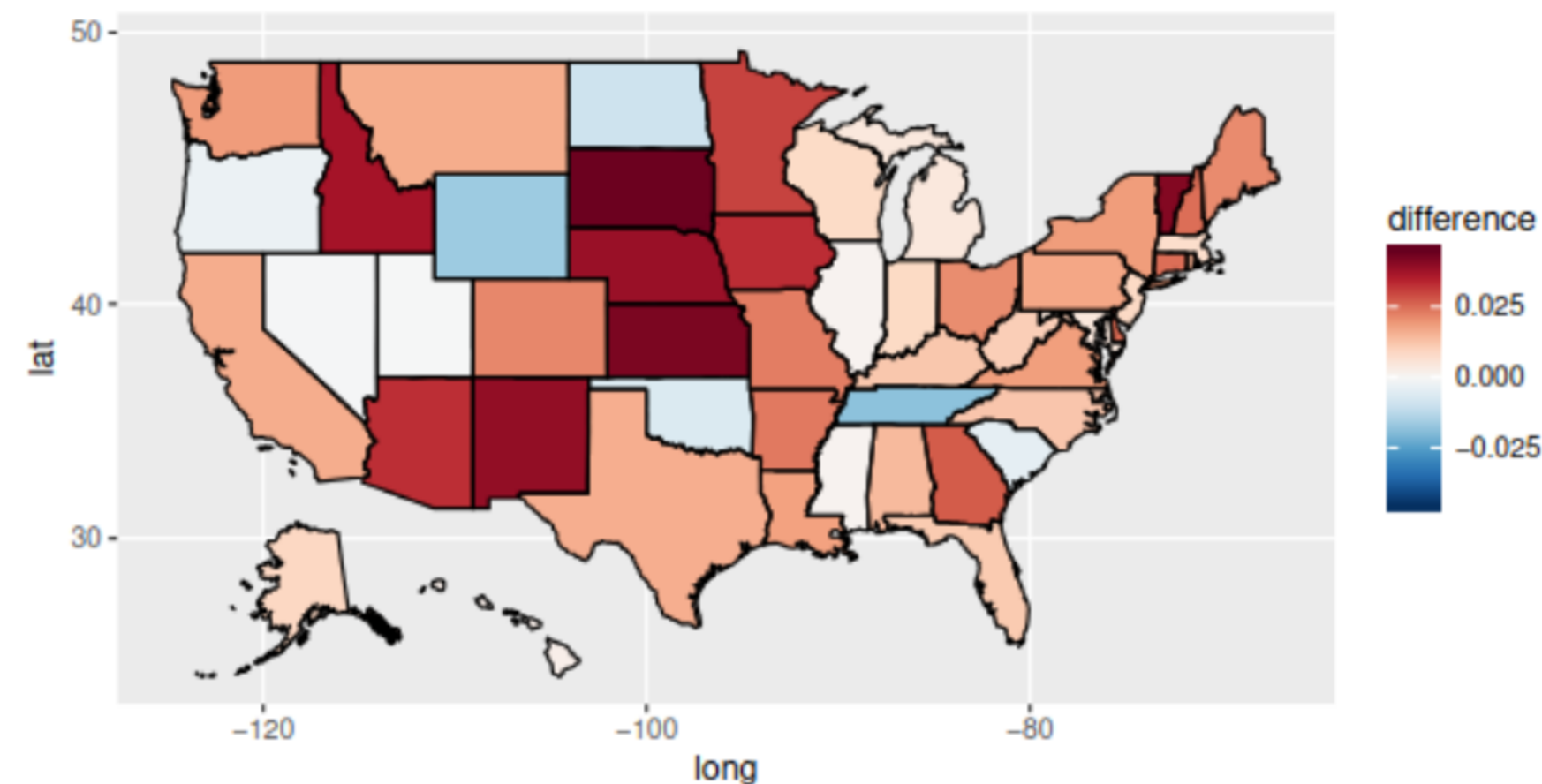
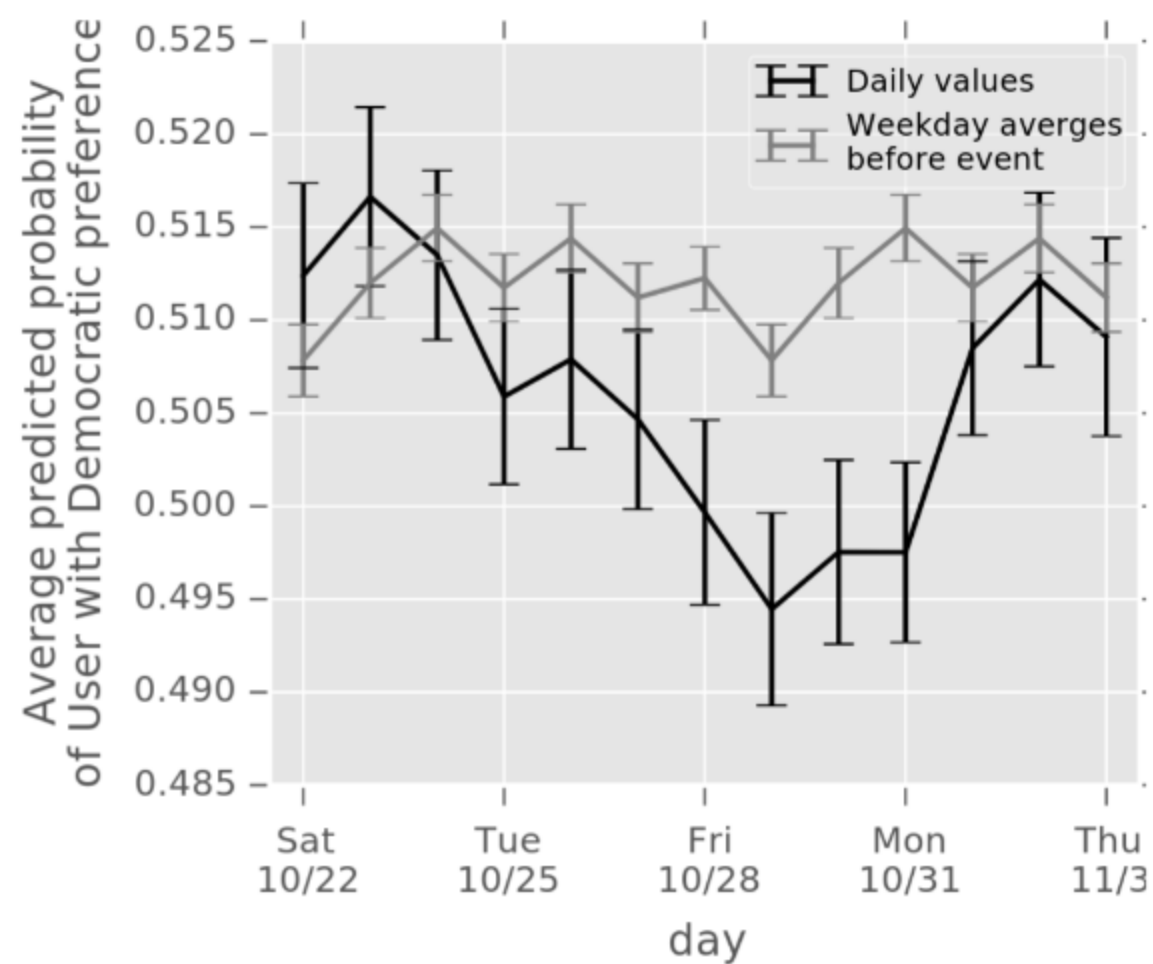


Figure 8: Impact of the 'Corney letter' at the state level.

# Prior Work

- Web browsing behavior can predict voting results
- Quantifying the 'Comey letter' (Comarella et al.)
- Social media referrals are the best signal



**Figure 8: Impact of the ‘Comey letter’ at the state level.**

# Prior Work

- Web browsing behavior can predict voting results
- Quantifying the 'Corney letter' (Comarella et al.)
- Social media referrals are the best signal
- Used large plaintext dataset

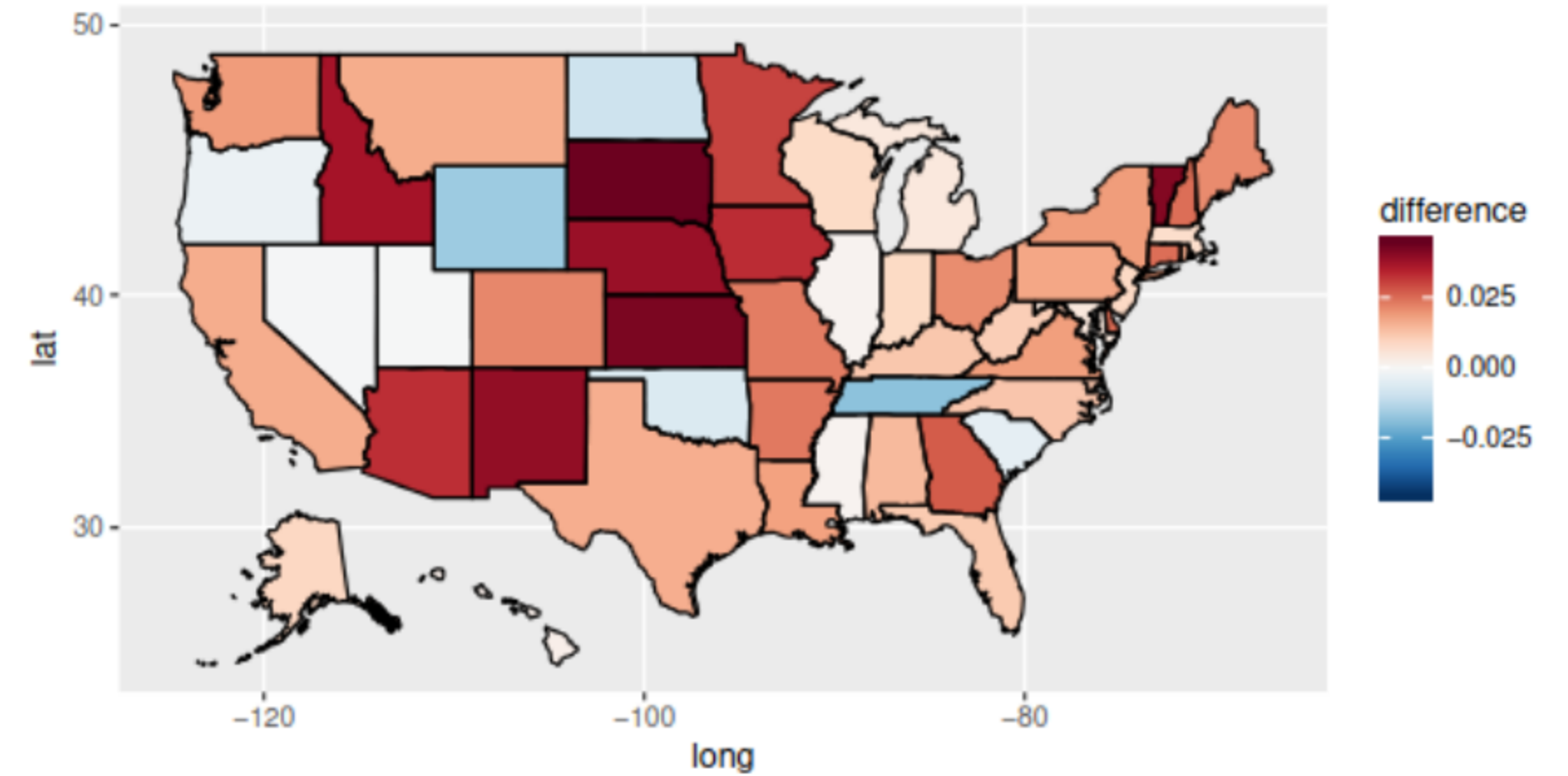
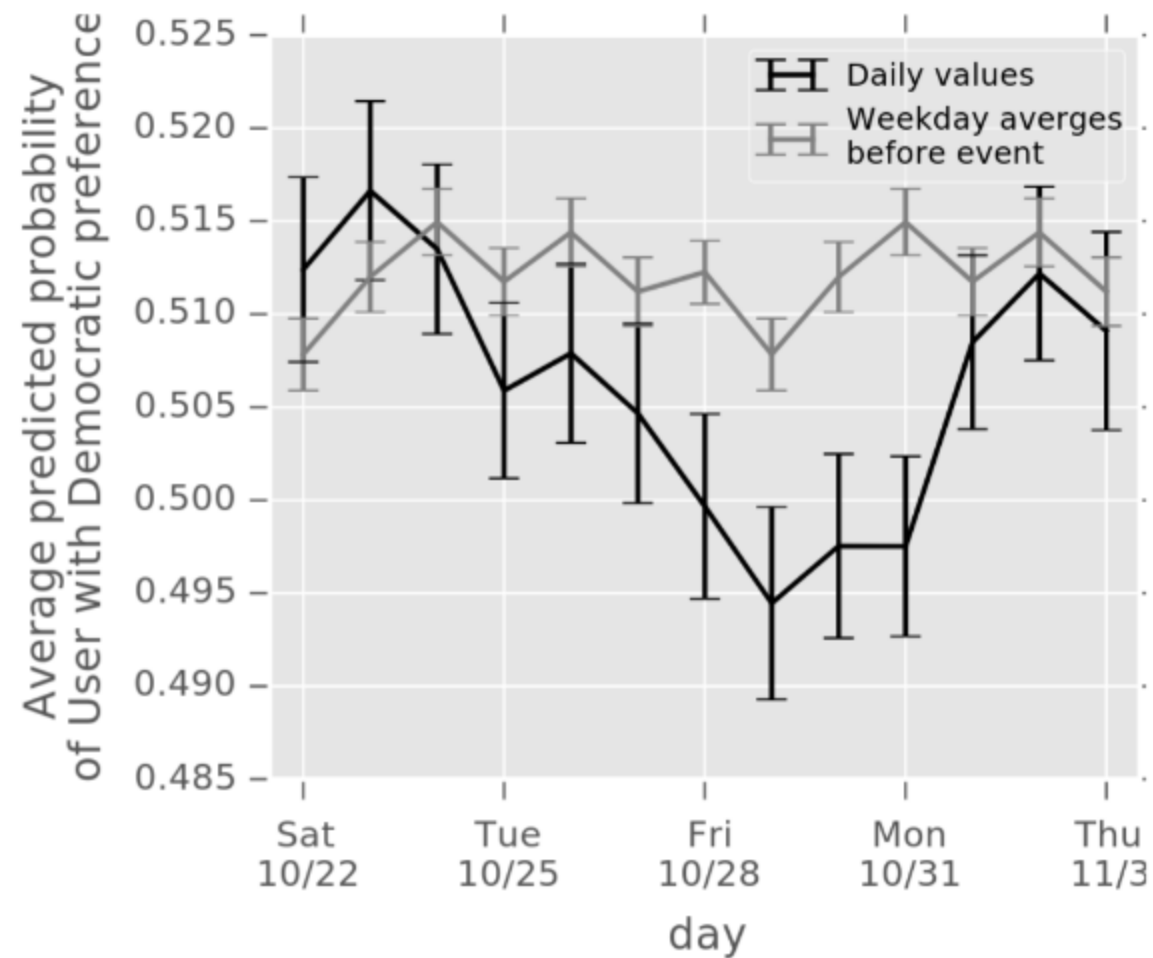


Figure 8: Impact of the 'Corney letter' at the state level.



# Two Approaches to Political Polling

# Two Approaches to Political Polling

## Traditional Polling

Slow

Expensive

Coarse-grained insights

# Two Approaches to Political Polling

## Traditional Polling

Slow

Expensive

Coarse-grained insights

VS

## Web Behavior Analysis

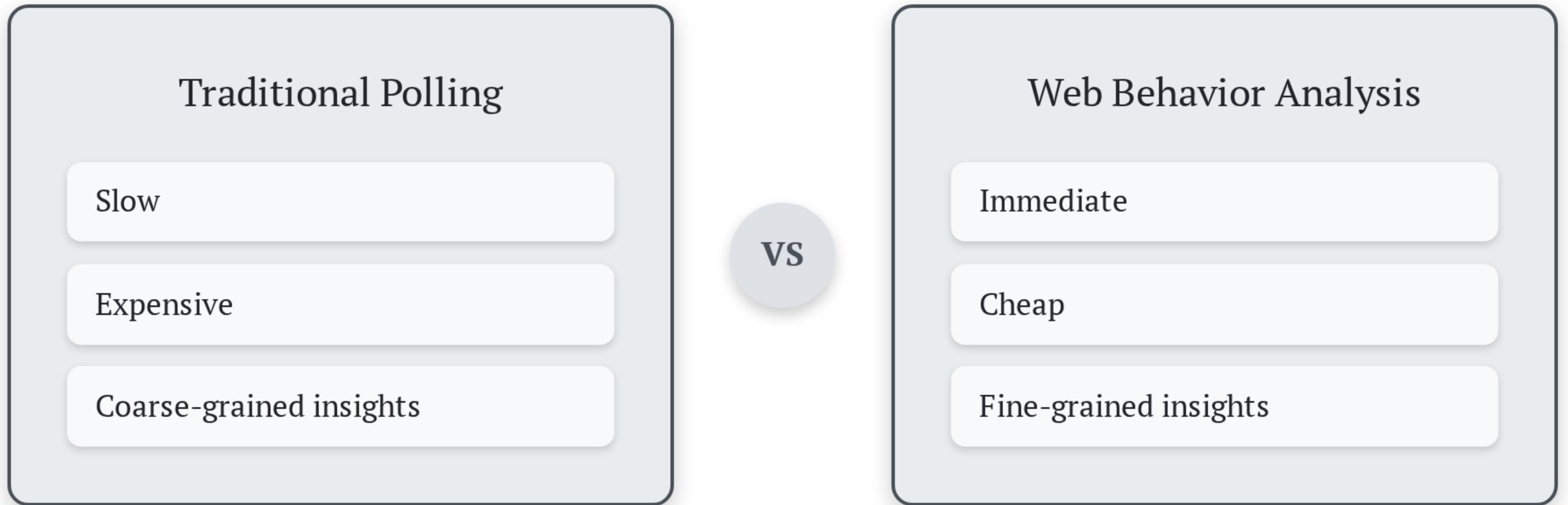
Immediate

Cheap

Fine-grained insights



# Two Approaches to Political Polling



What about privacy?

# Our Contributions

# Our Contributions

- We built a system for securely predicting political preferences from web browsing data

# Our Contributions

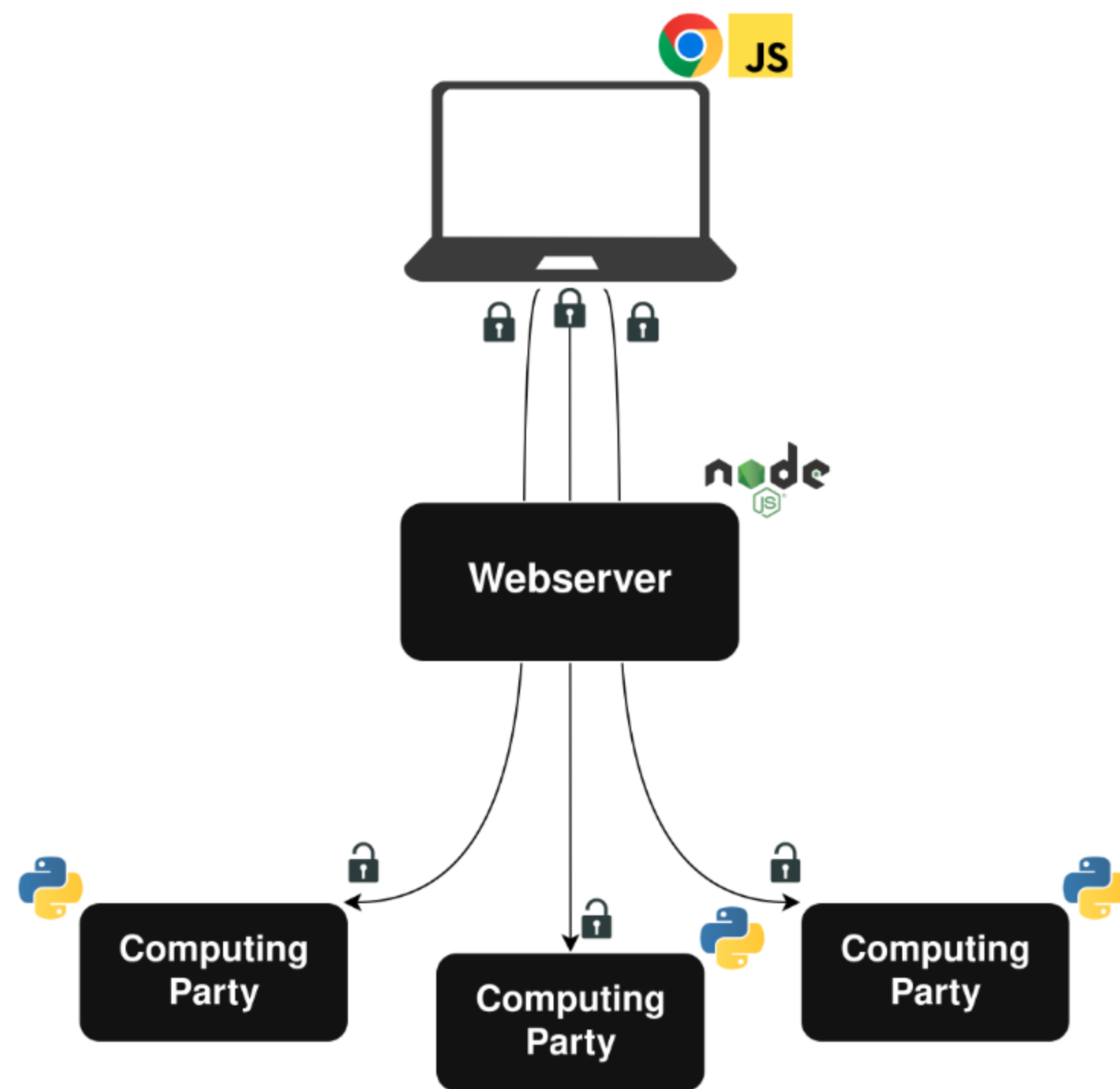
- We built a system for securely predicting political preferences from web browsing data
- We collected and analyzed data from almost 8000 unique users

# Our Contributions

- We built a system for securely predicting political preferences from web browsing data
- We collected and analyzed data from almost 8000 unique users
- All analysis took place under MPC

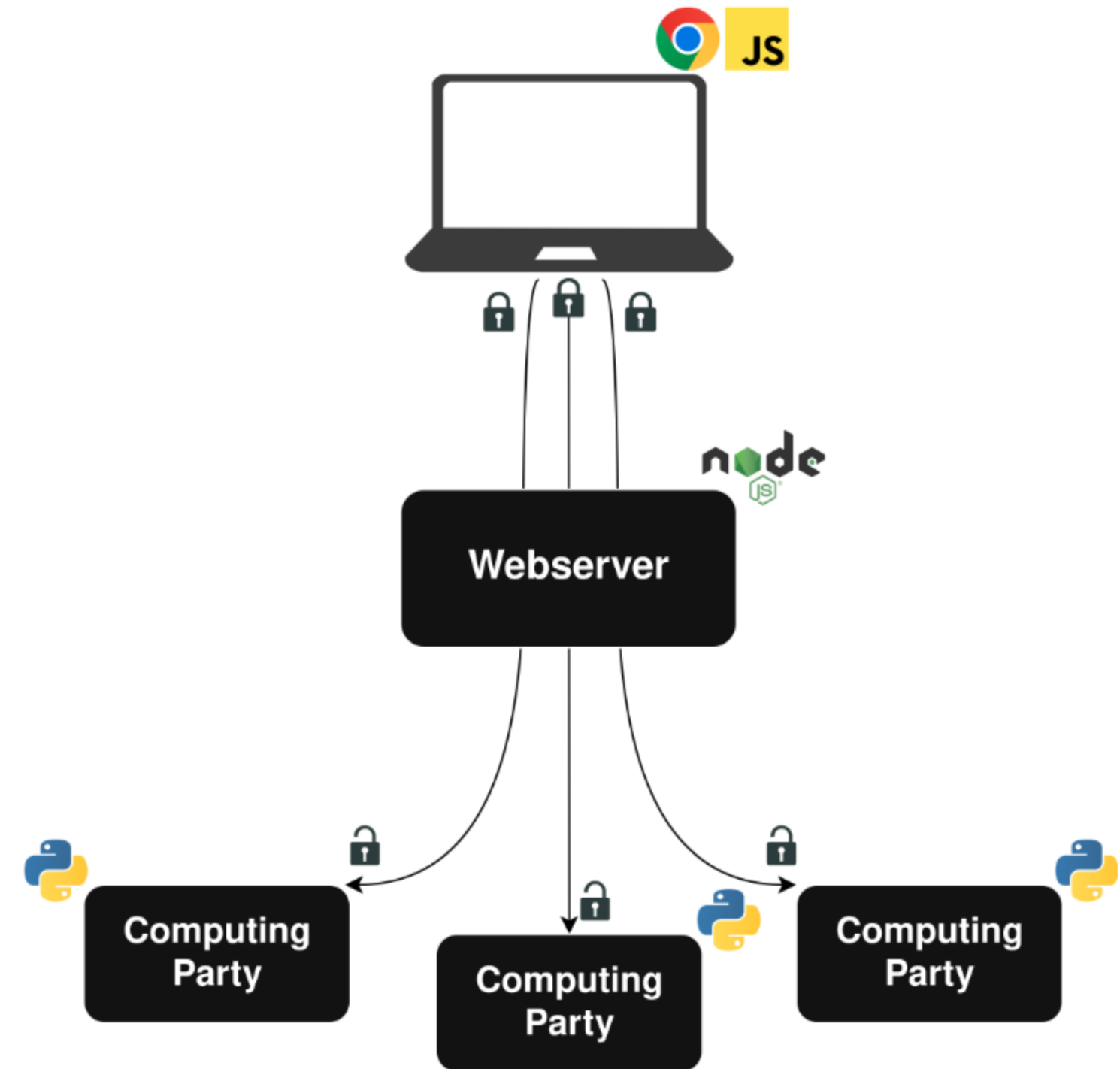


# System Design



# System Design

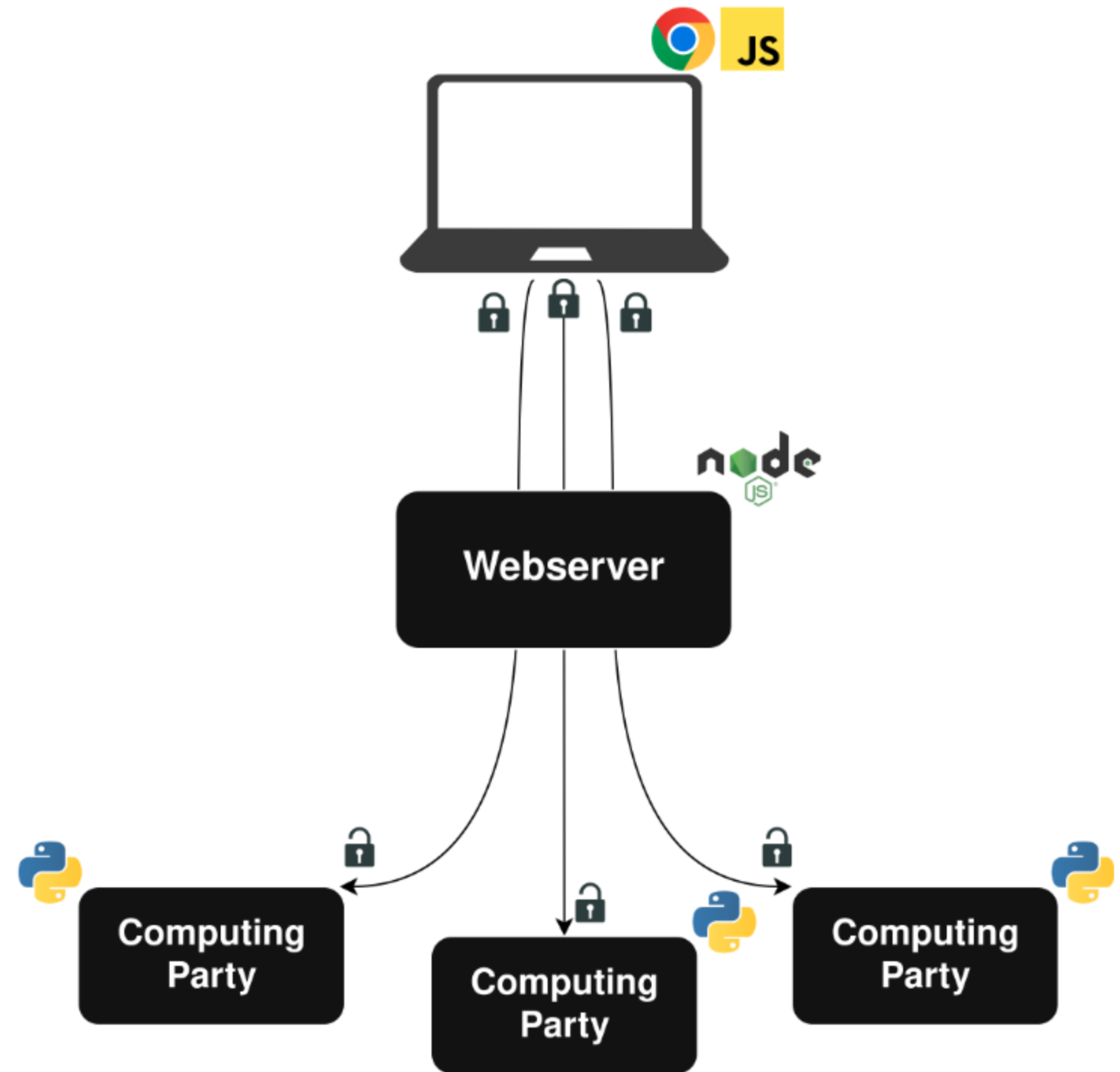
Client Plugin



# System Design

Client Plugin

Webserver



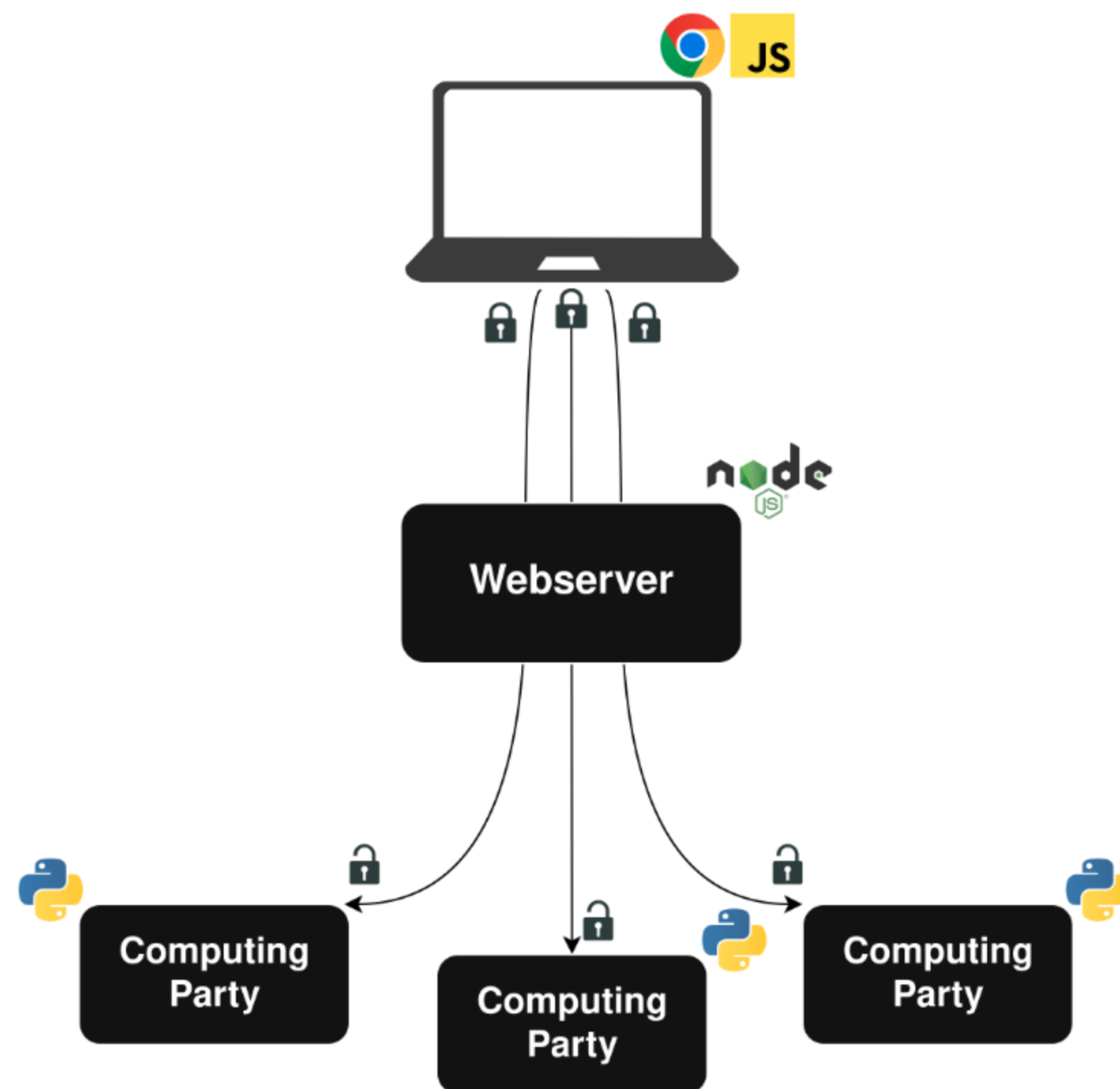


# System Design

Client Plugin

Webserver

MPC Backend

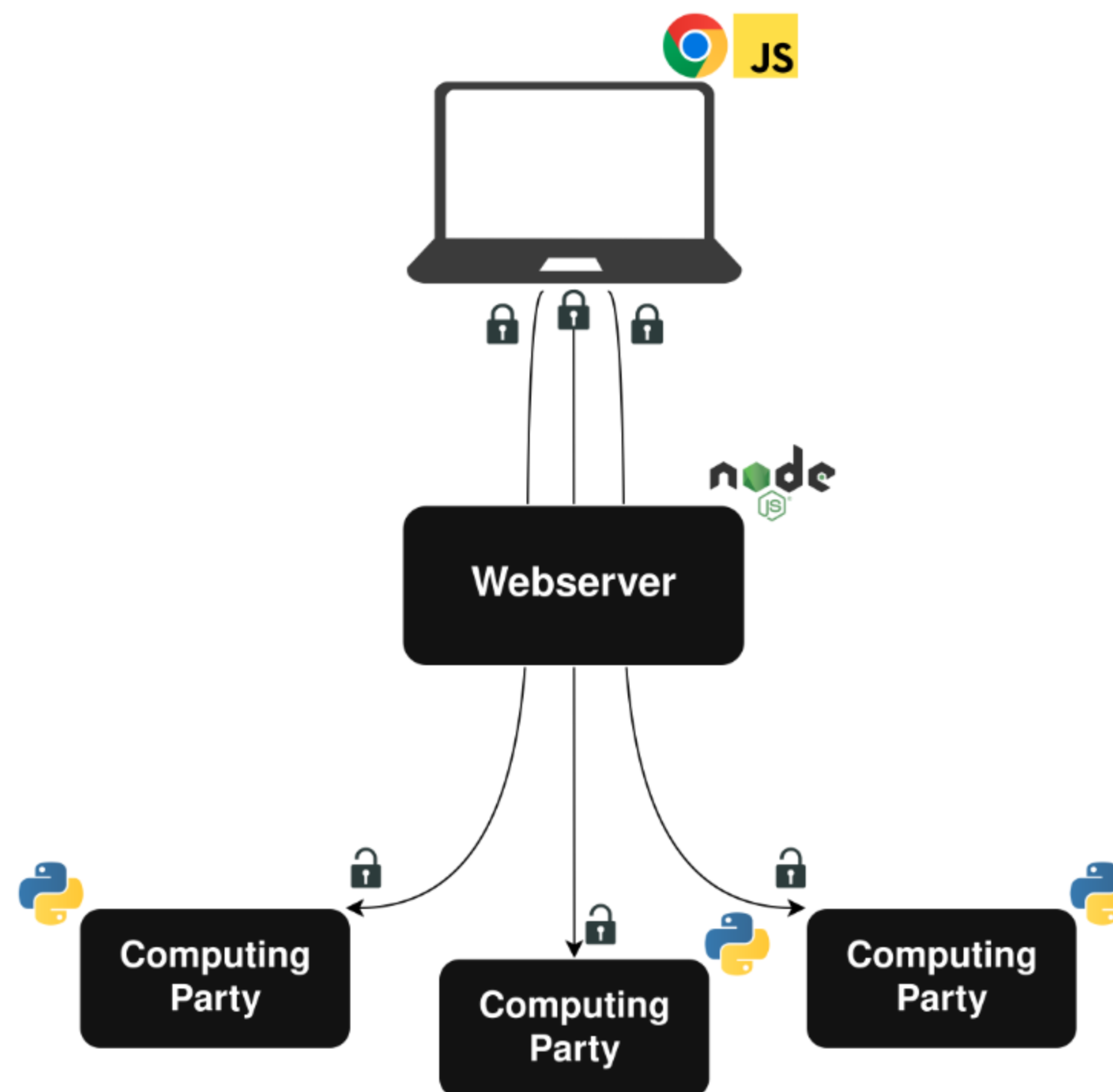


# System Design

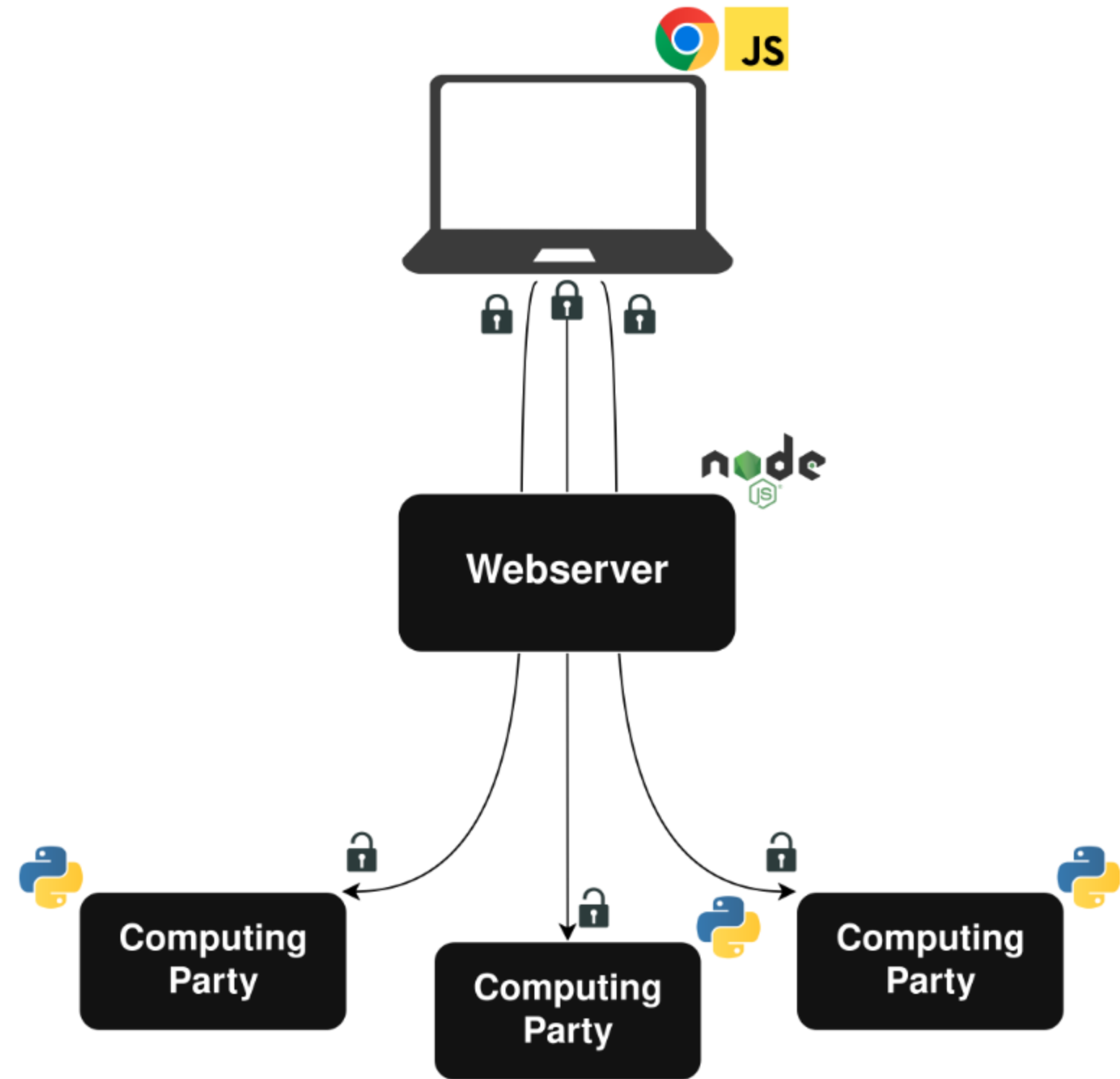
Client Plugin

Webserver

MPC Backend

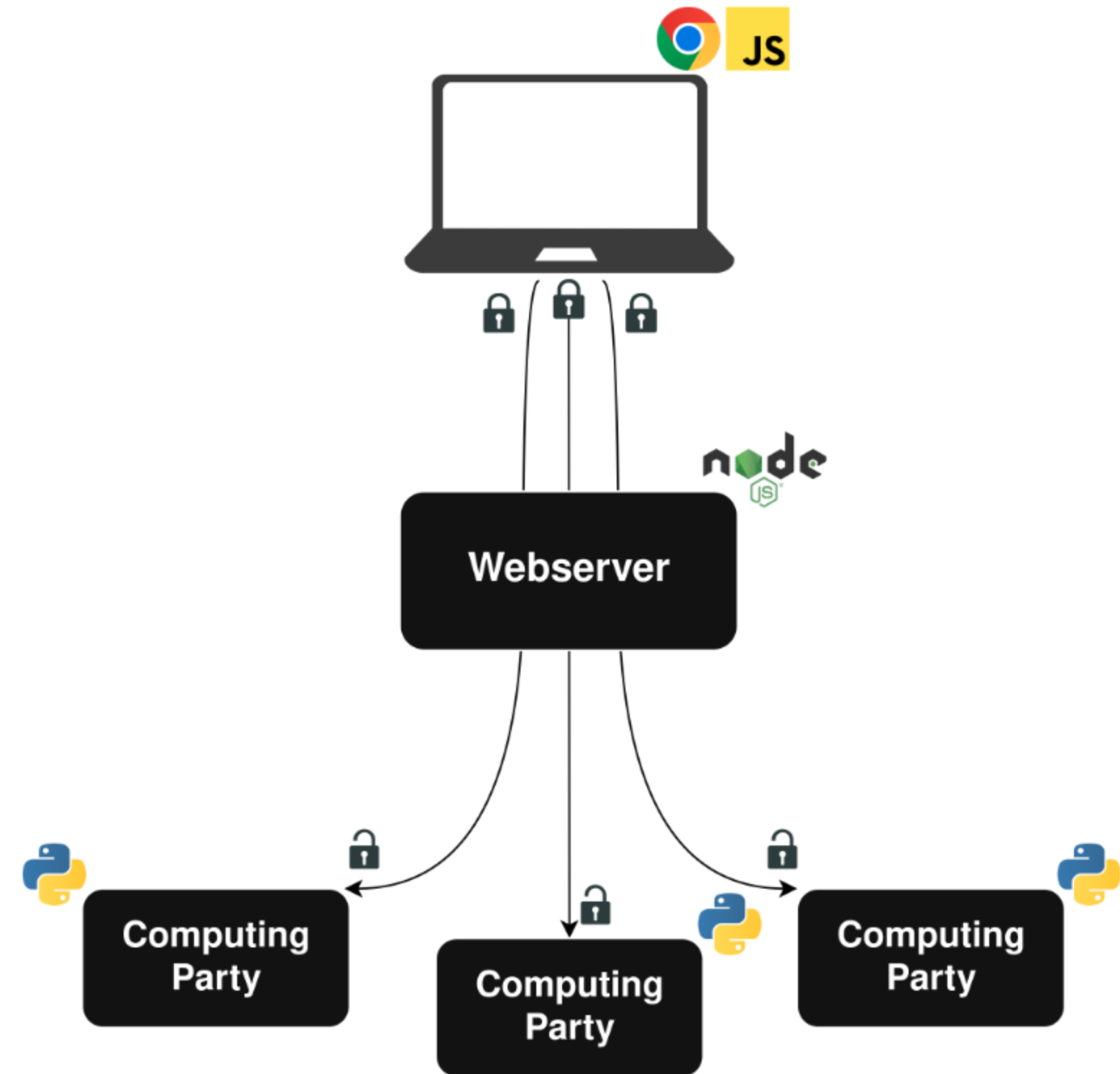


# Client Plugin



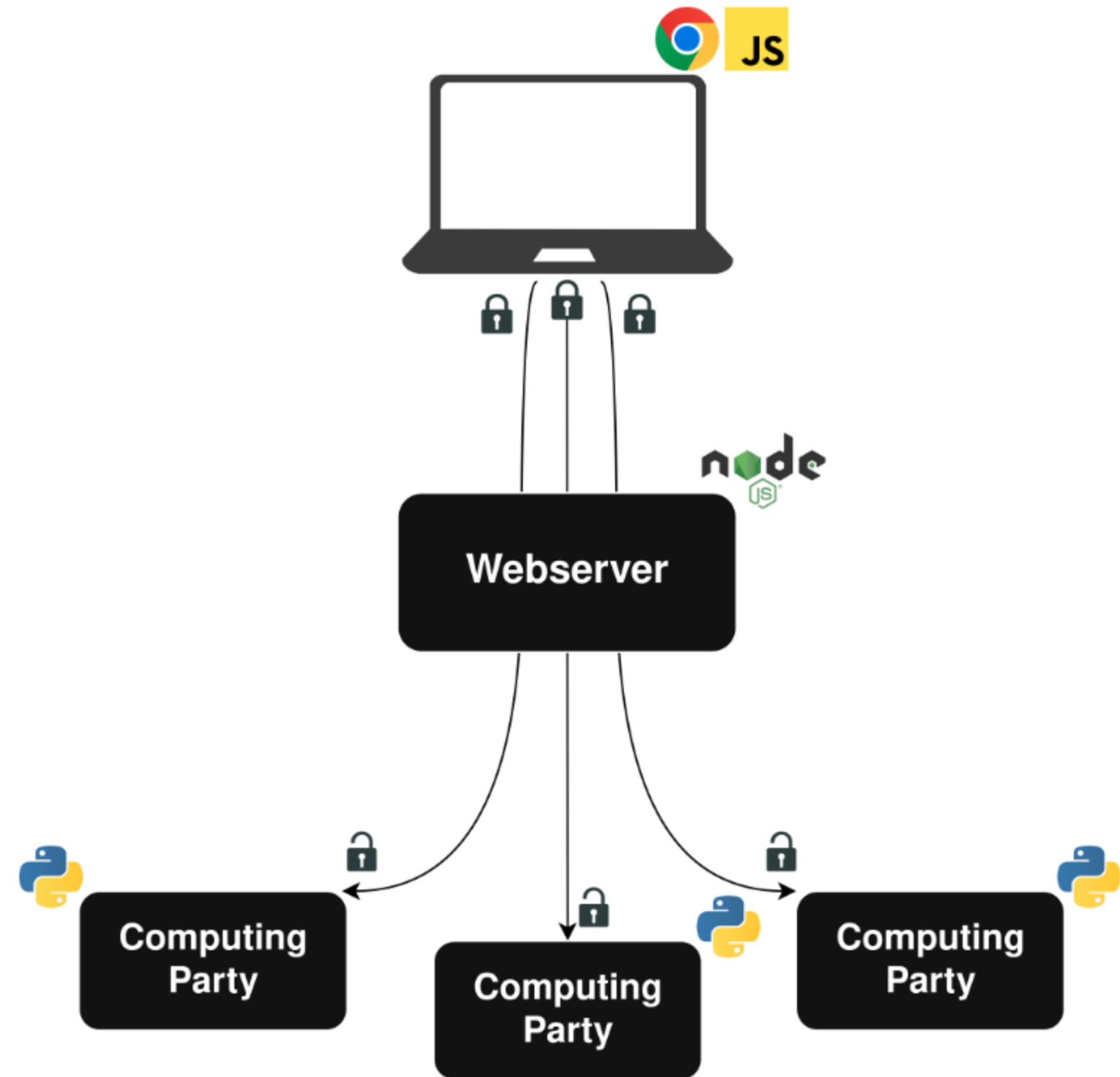
# Client Plugin

- Custom-built Chrome plugin to monitor browsing



# Client Plugin

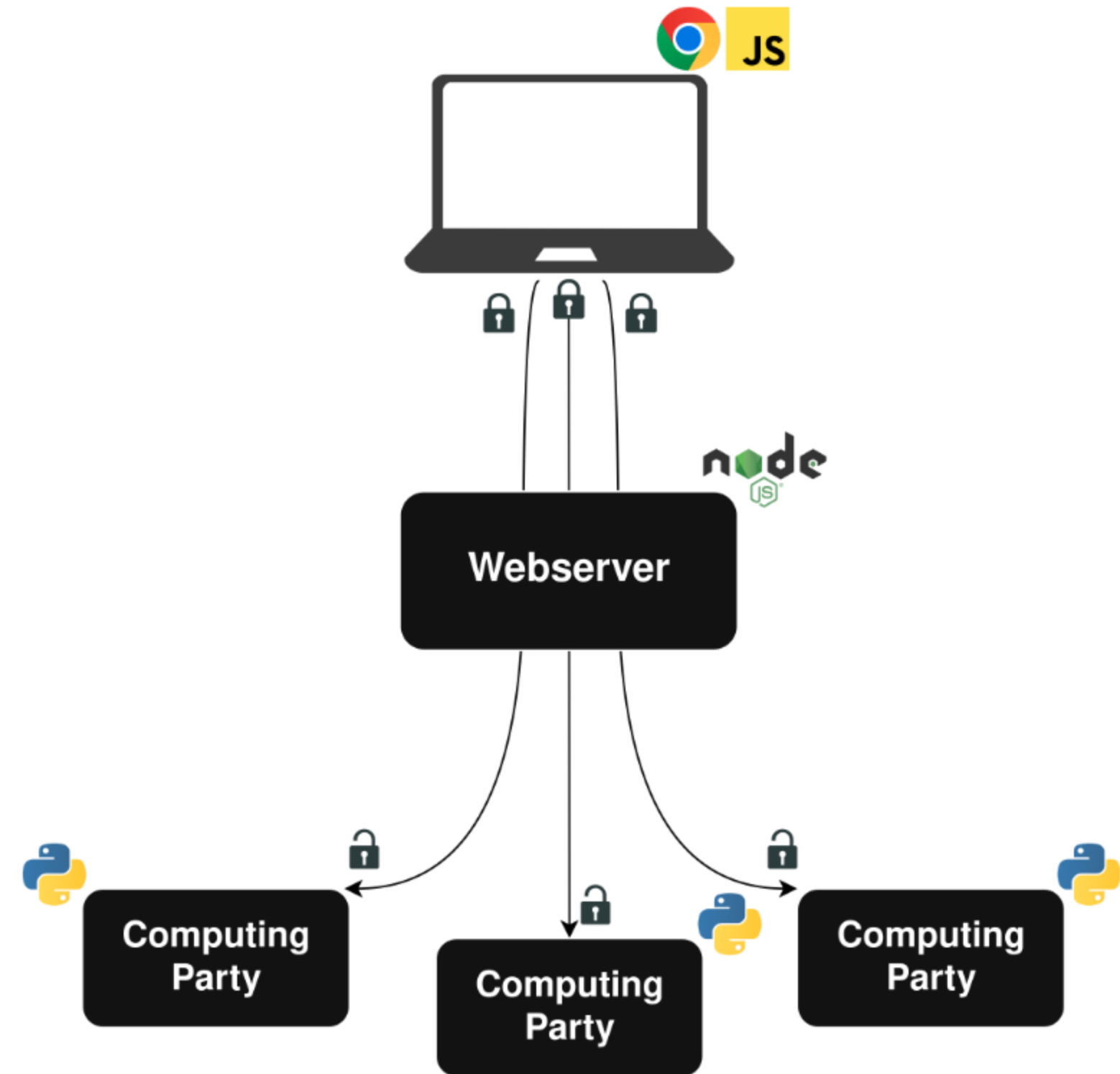
- Custom-built Chrome plugin to monitor browsing
- Daily data uploads of secret-shared histograms





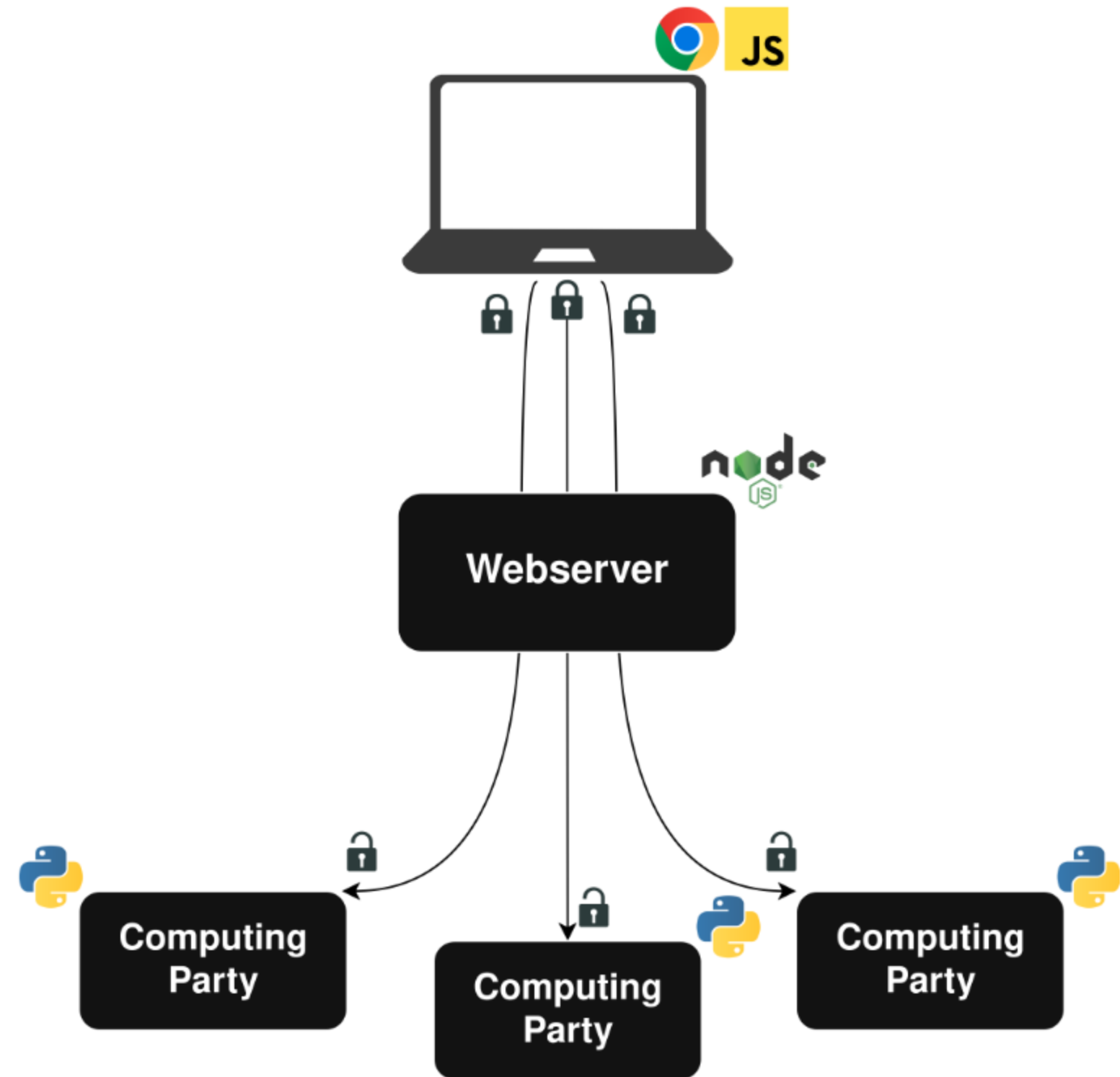
# Client Plugin

- Custom-built Chrome plugin to monitor browsing
- Daily data uploads of secret-shared histograms
- Client-side secret sharing and encryption

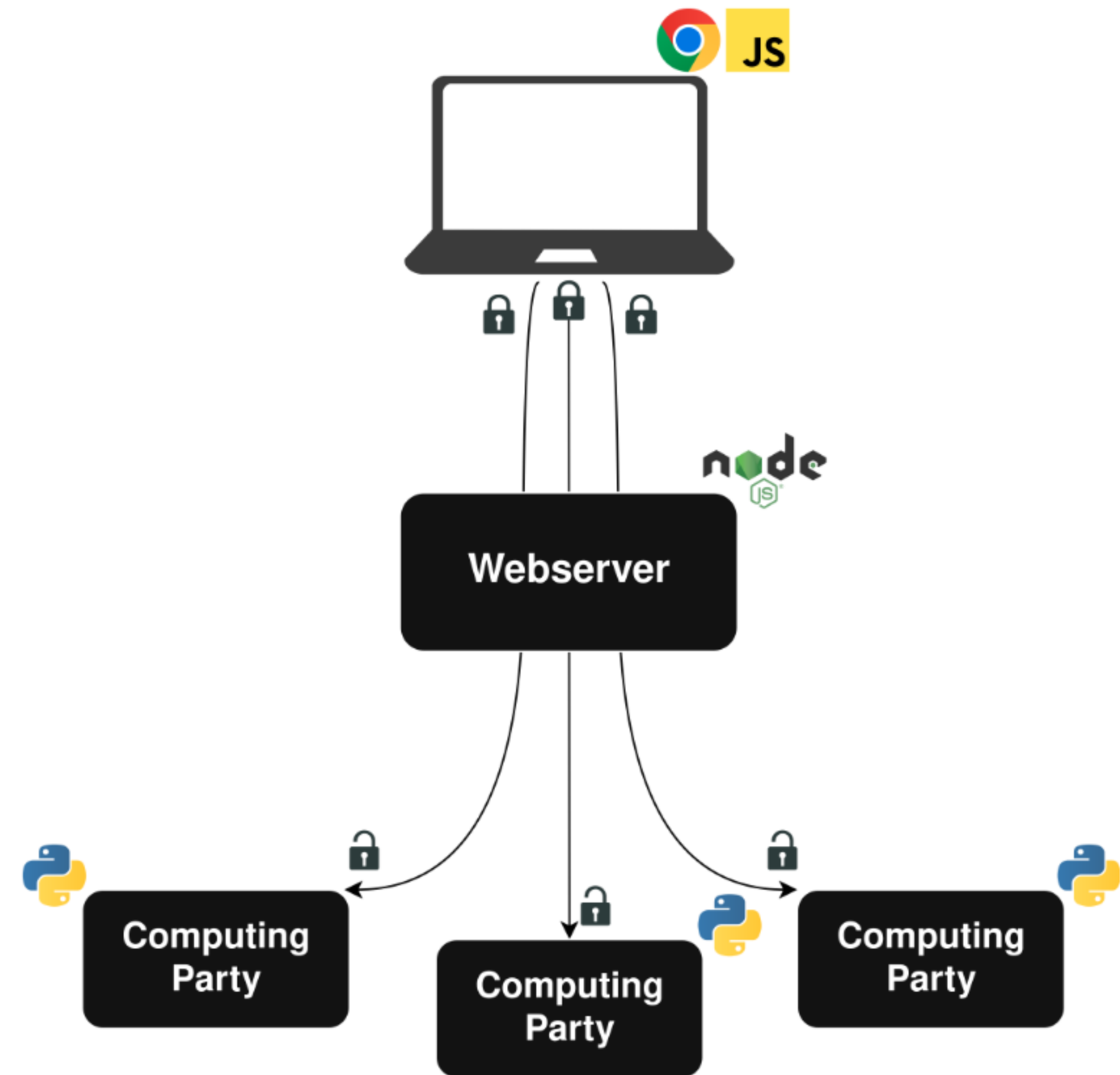


# Client Plugin

- Custom-built Chrome plugin to monitor browsing
- Daily data uploads of secret-shared histograms
- Client-side secret sharing and encryption
- Implementation is open source



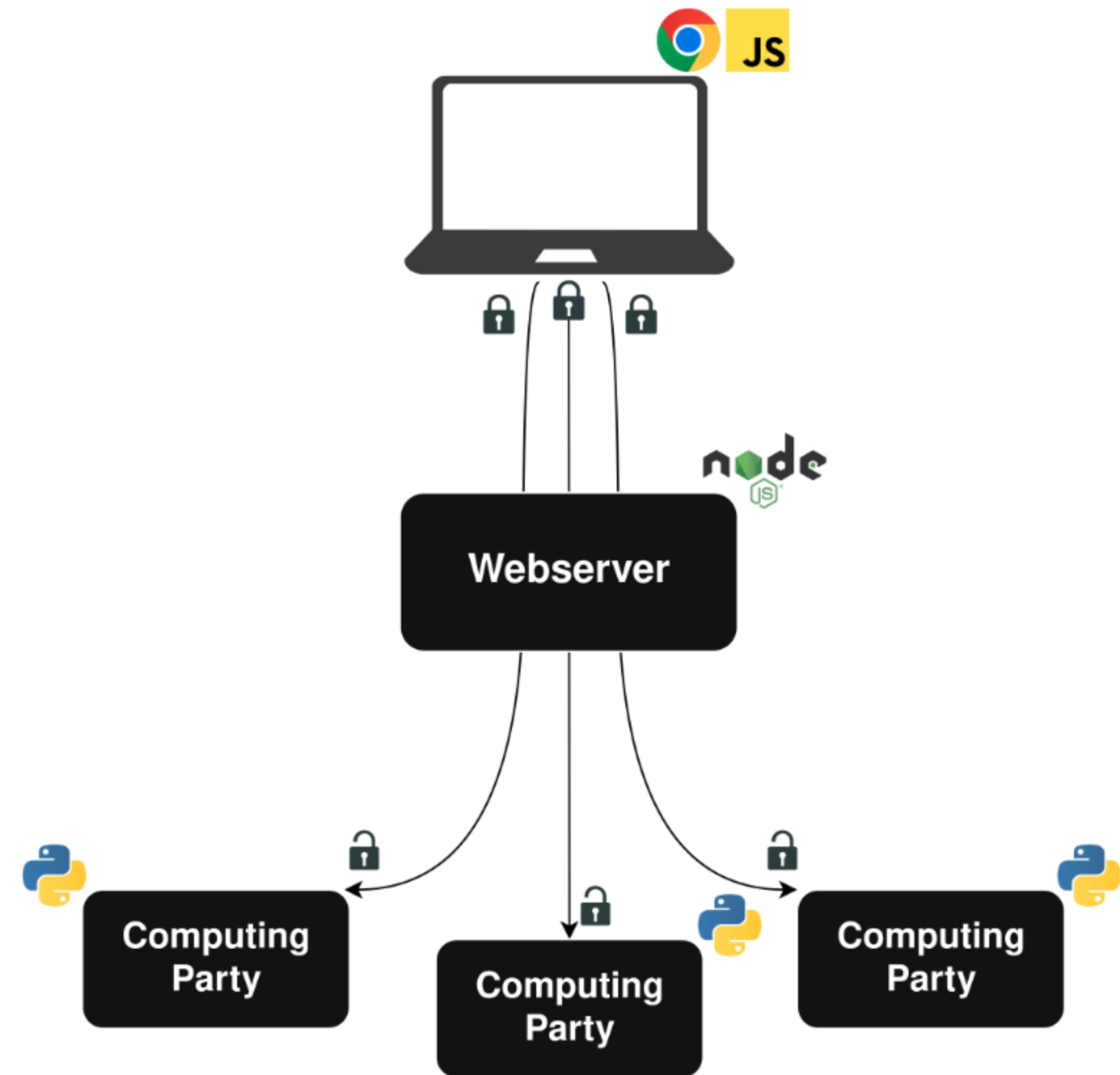
# Webserver





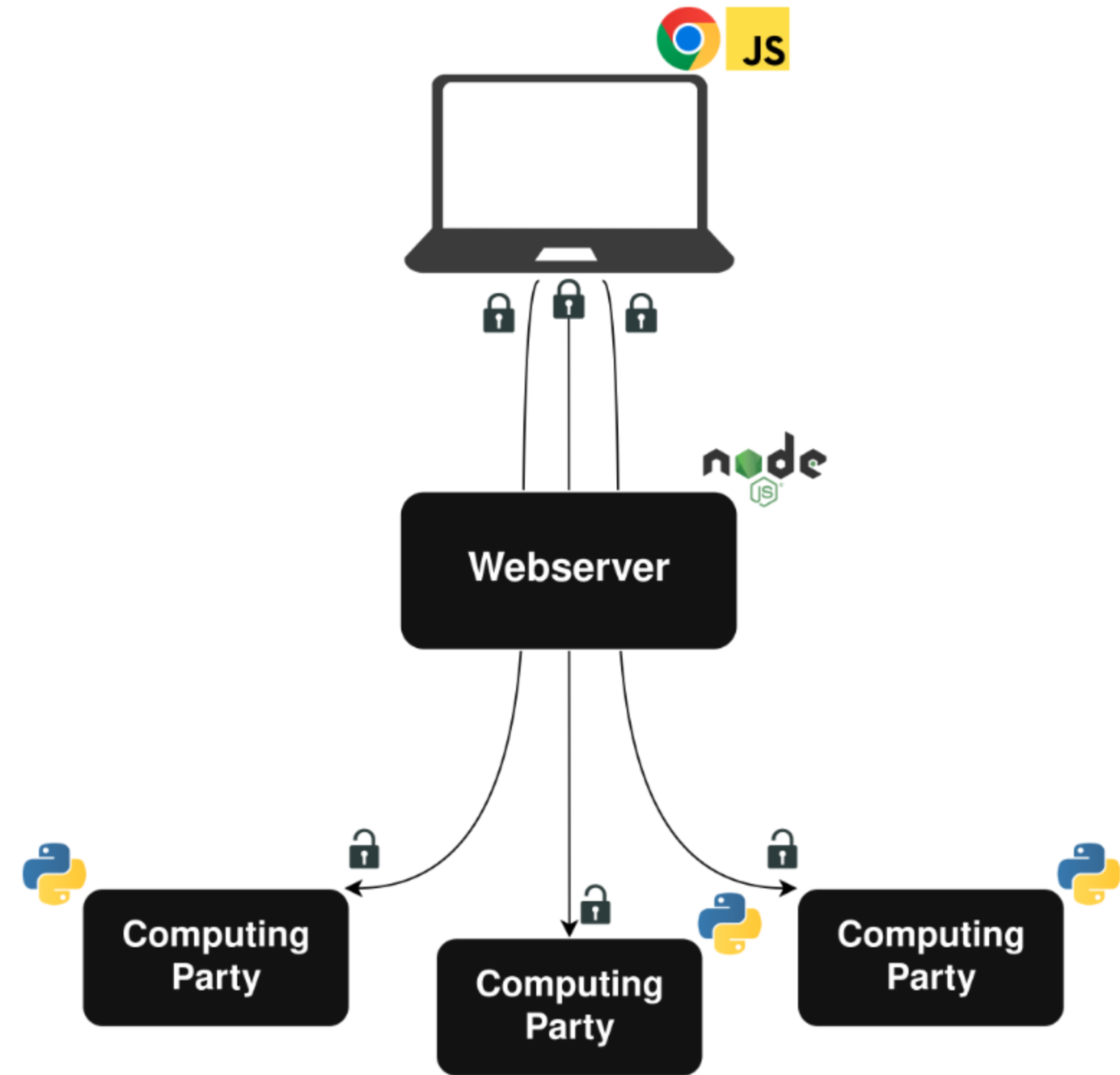
# Webserver

- Simplifies interaction with clients



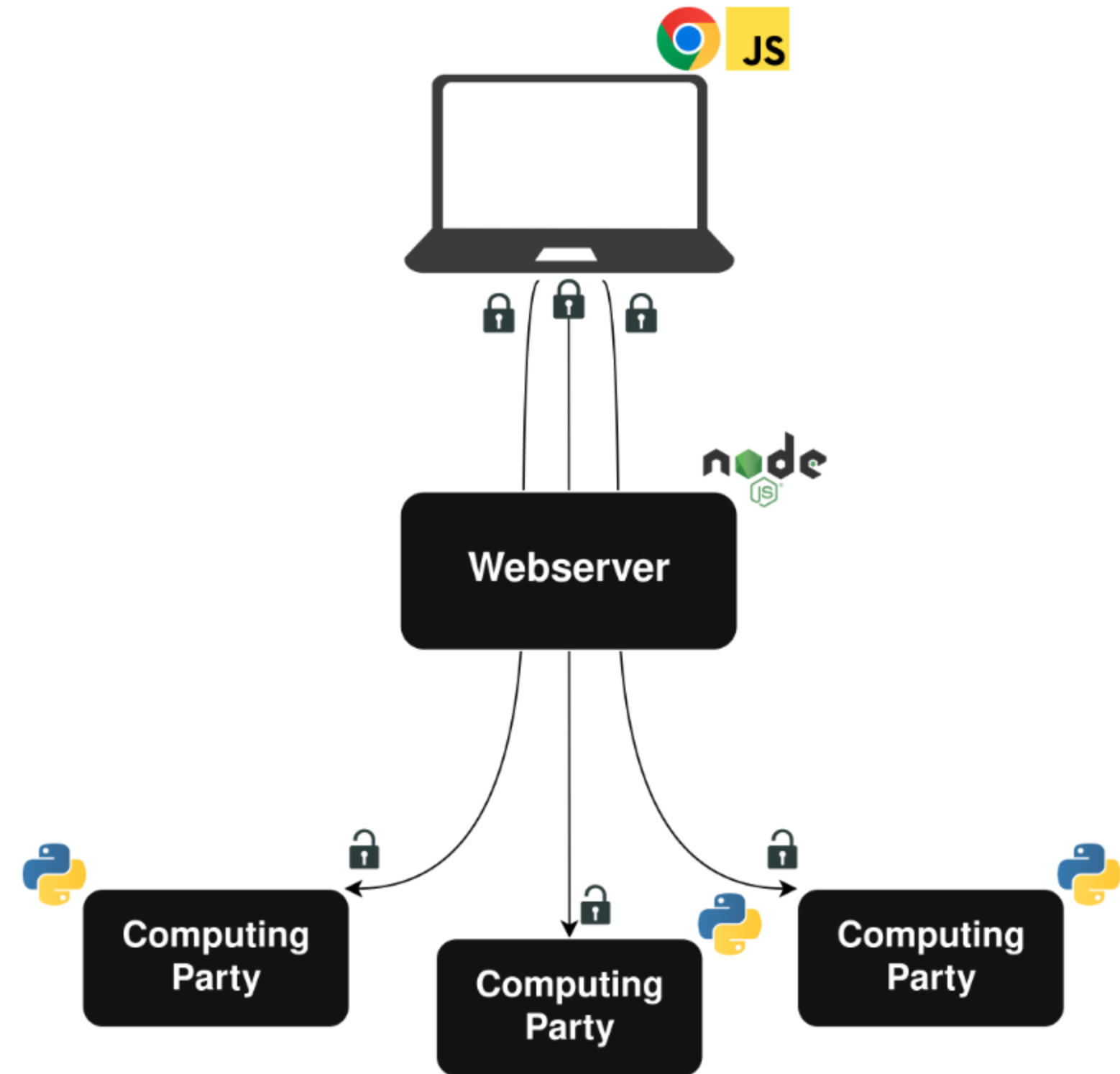
# Webserver

- Simplifies interaction with clients
- Collects basic metadata

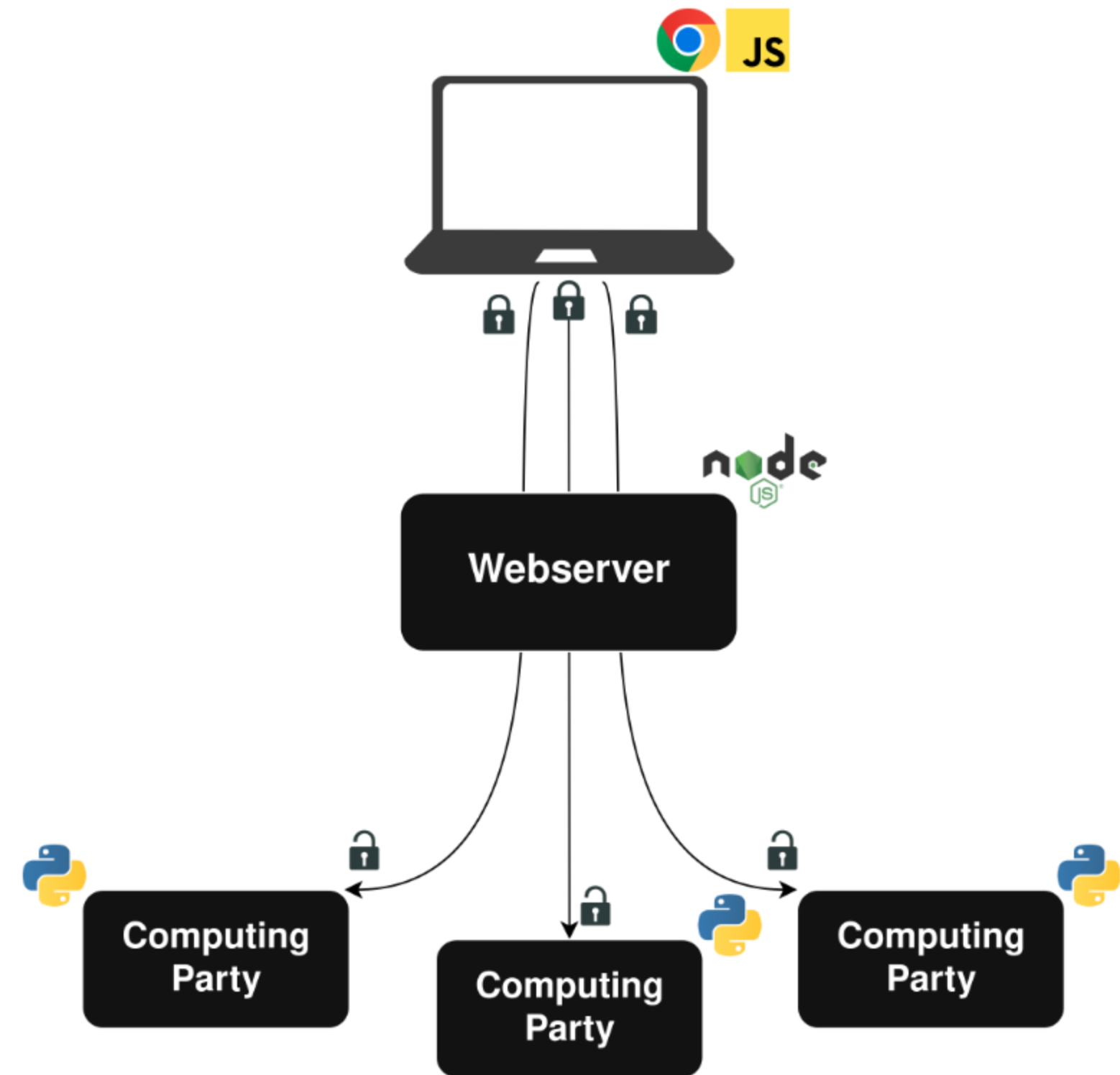


# Webserver

- Simplifies interaction with clients
- Collects basic metadata
- Never sees any private data

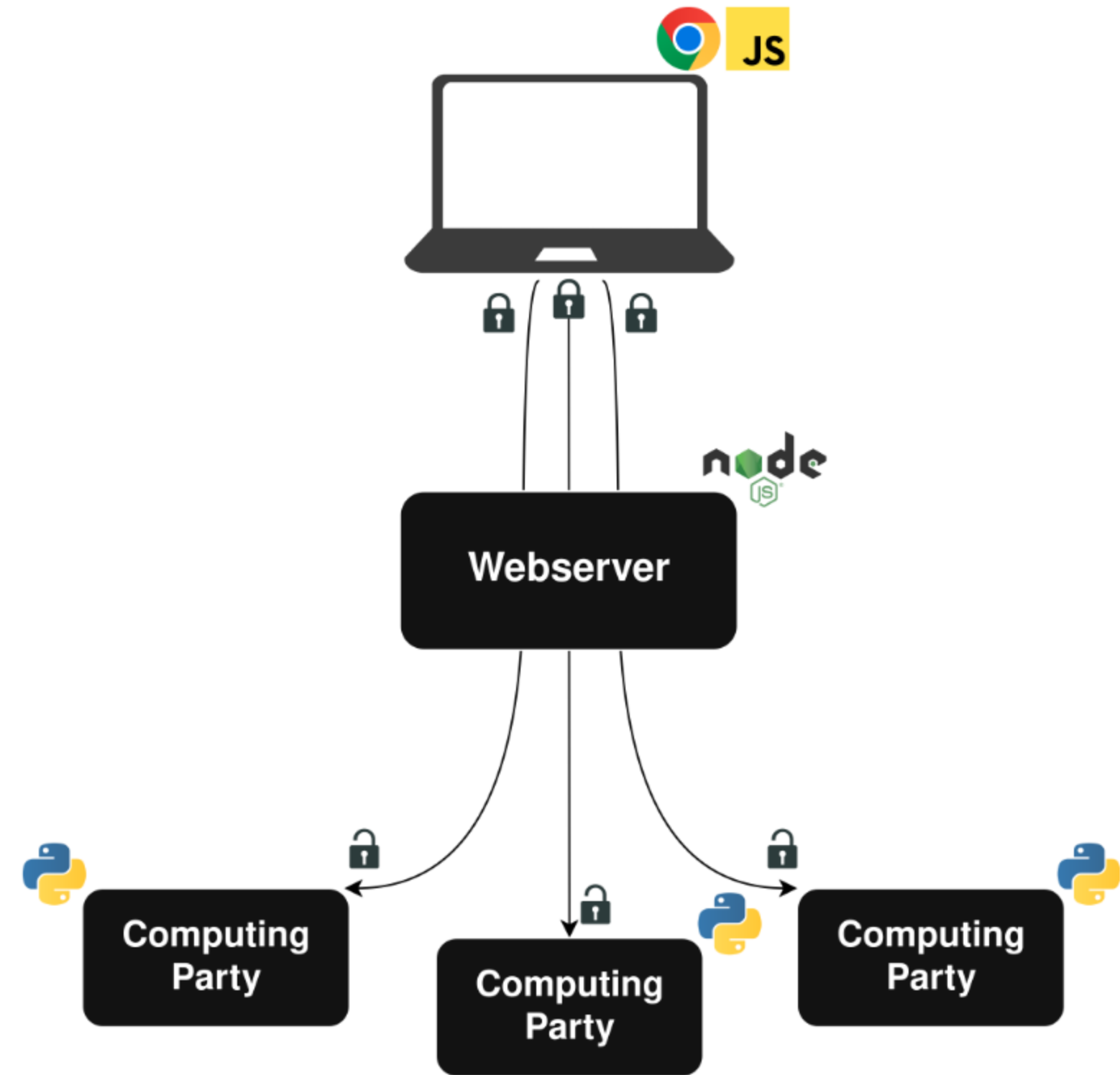


# MPC Backend



# MPC Backend

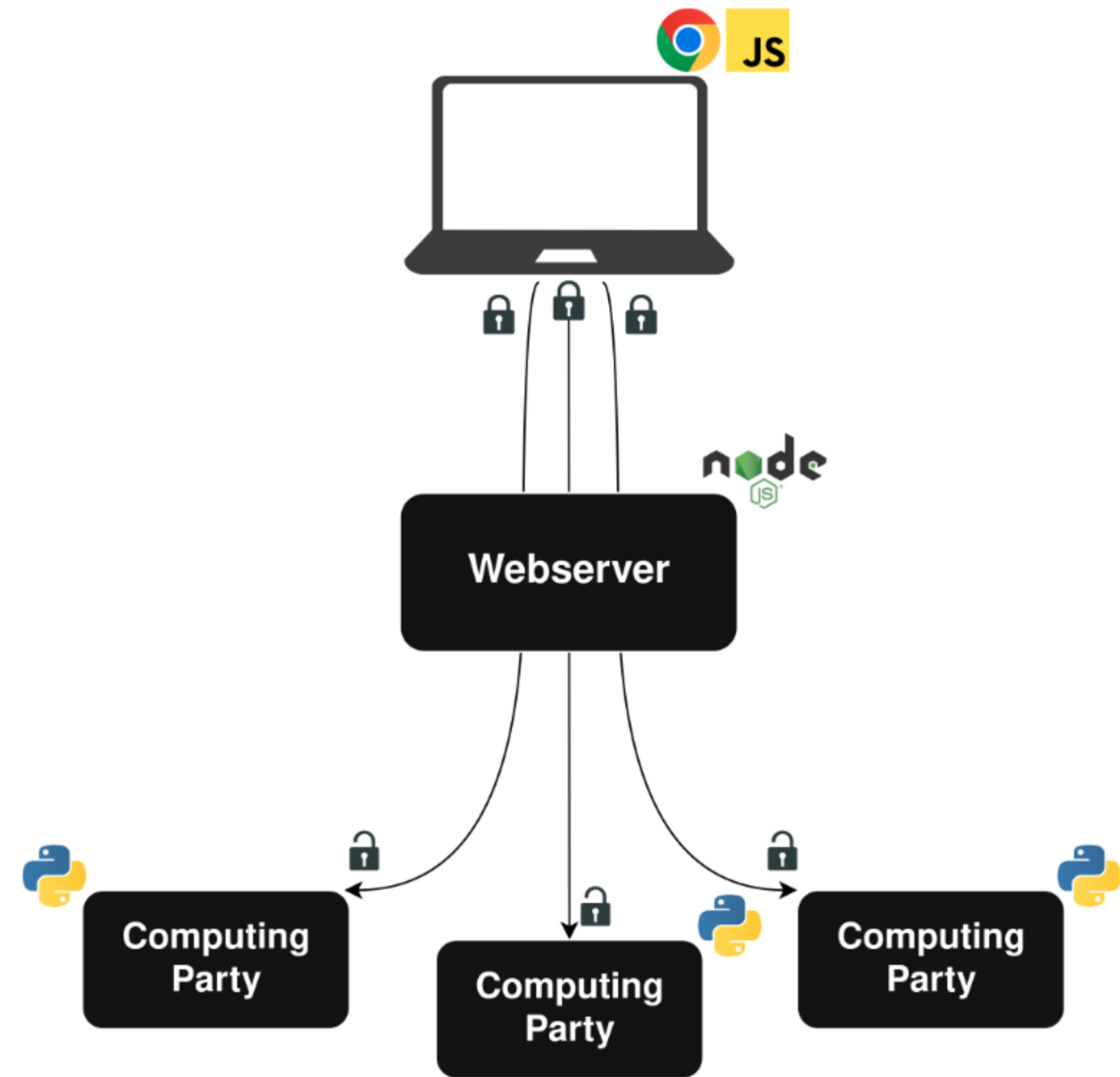
- Three party computation with an honest majority





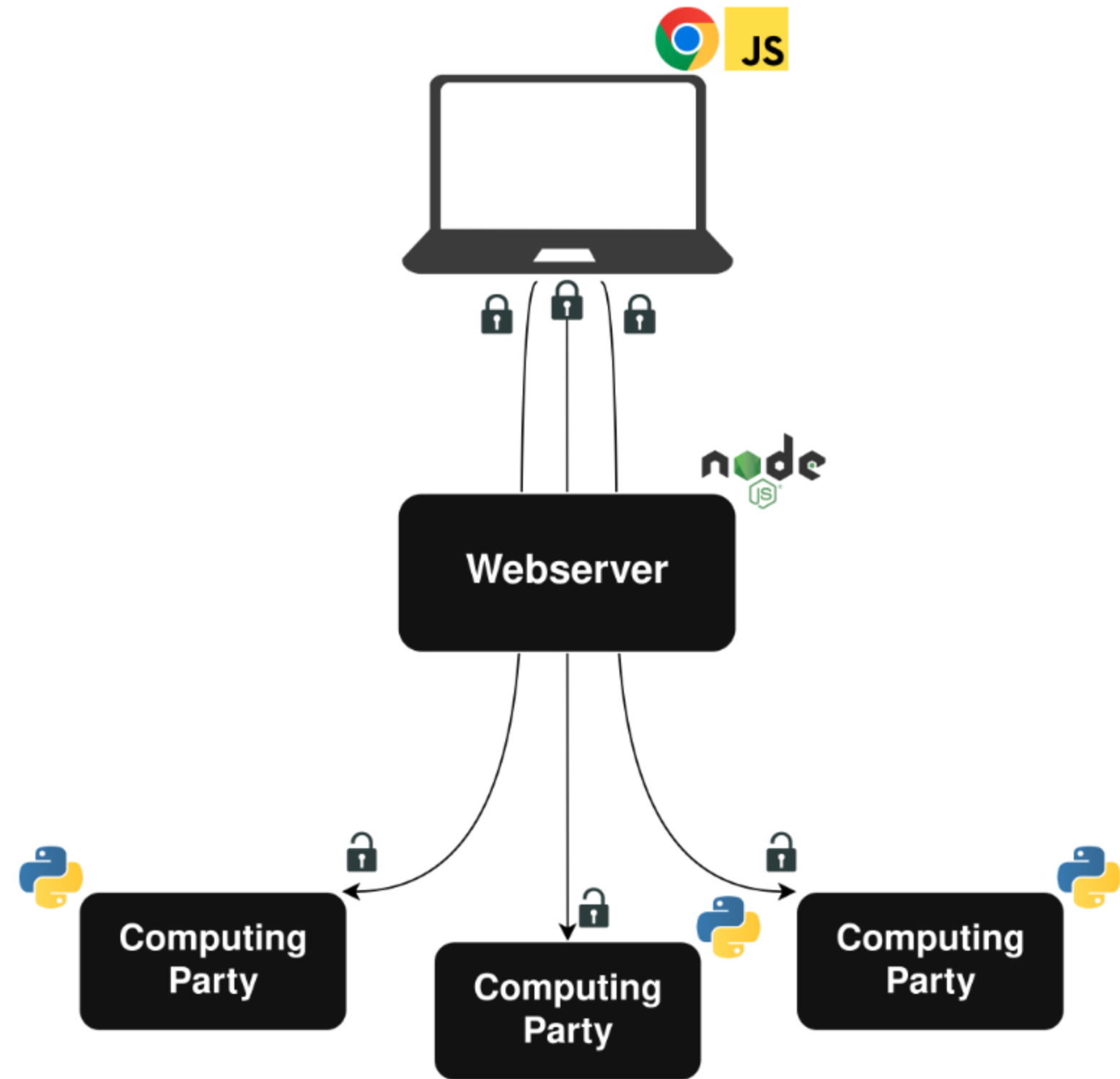
# MPC Backend

- Three party computation with an honest majority
- We used and augmented the CrypTen library



# MPC Backend

- Three party computation with an honest majority
- We used and augmented the CrypTen library
- We implemented an algorithm for LLP under MPC



# The Learning Process



# The Learning Process

We're working with the learning from label proportions (LLP) problem.

# The Learning Process

We're working with the learning from label proportions (LLP) problem.

- The first implementation under MPC of an LLP model

# The Learning Process

We're working with the learning from label proportions (LLP) problem.

- The first implementation under MPC of an LLP model
- Mix of out-of-the-box components and custom implementations

# Learning from Label Proportions (LLP)

# Learning from Label Proportions (LLP)

- Each user uploads an *unlabeled* 1,034-element vector every day



# Learning from Label Proportions (LLP)

- Each user uploads an *unlabeled* 1,034-element vector every day
  - Number of visits to the top 517 sites

# Learning from Label Proportions (LLP)

- Each user uploads an *unlabeled* 1,034-element vector every day
  - Number of visits to the top 517 sites
  - Number of times referred to the top 517 sites

# Learning from Label Proportions (LLP)

- Each user uploads an *unlabeled* 1,034-element vector every day
  - Number of visits to the top 517 sites
  - Number of times referred to the top 517 sites
- Unlabeled vectors are grouped by state

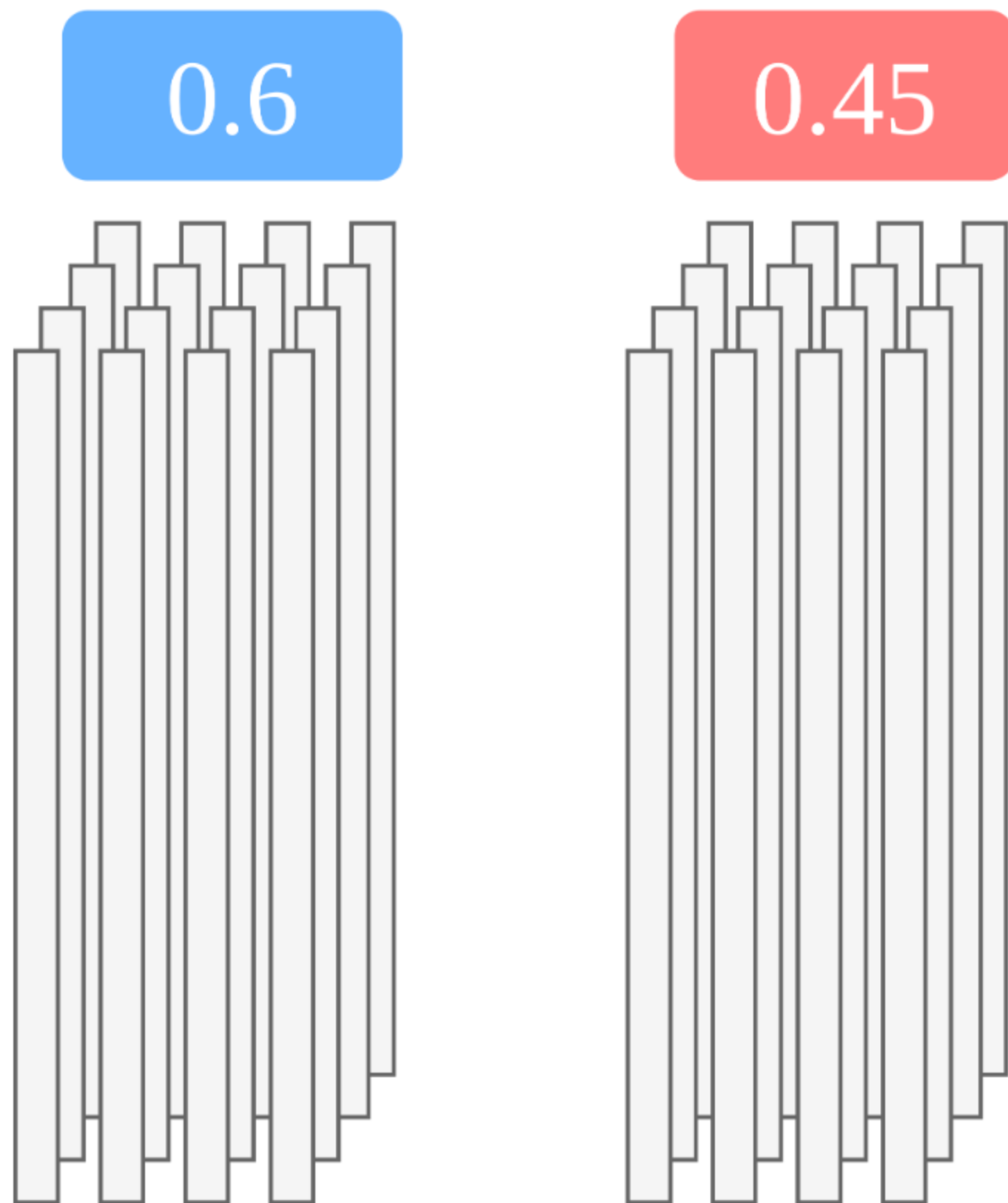


# Learning from Label Proportions (LLP)

- Each user uploads an *unlabeled* 1,034-element vector every day
  - Number of visits to the top 517 sites
  - Number of times referred to the top 517 sites
- Unlabeled vectors are grouped by state
- Each state has a ground-truth label

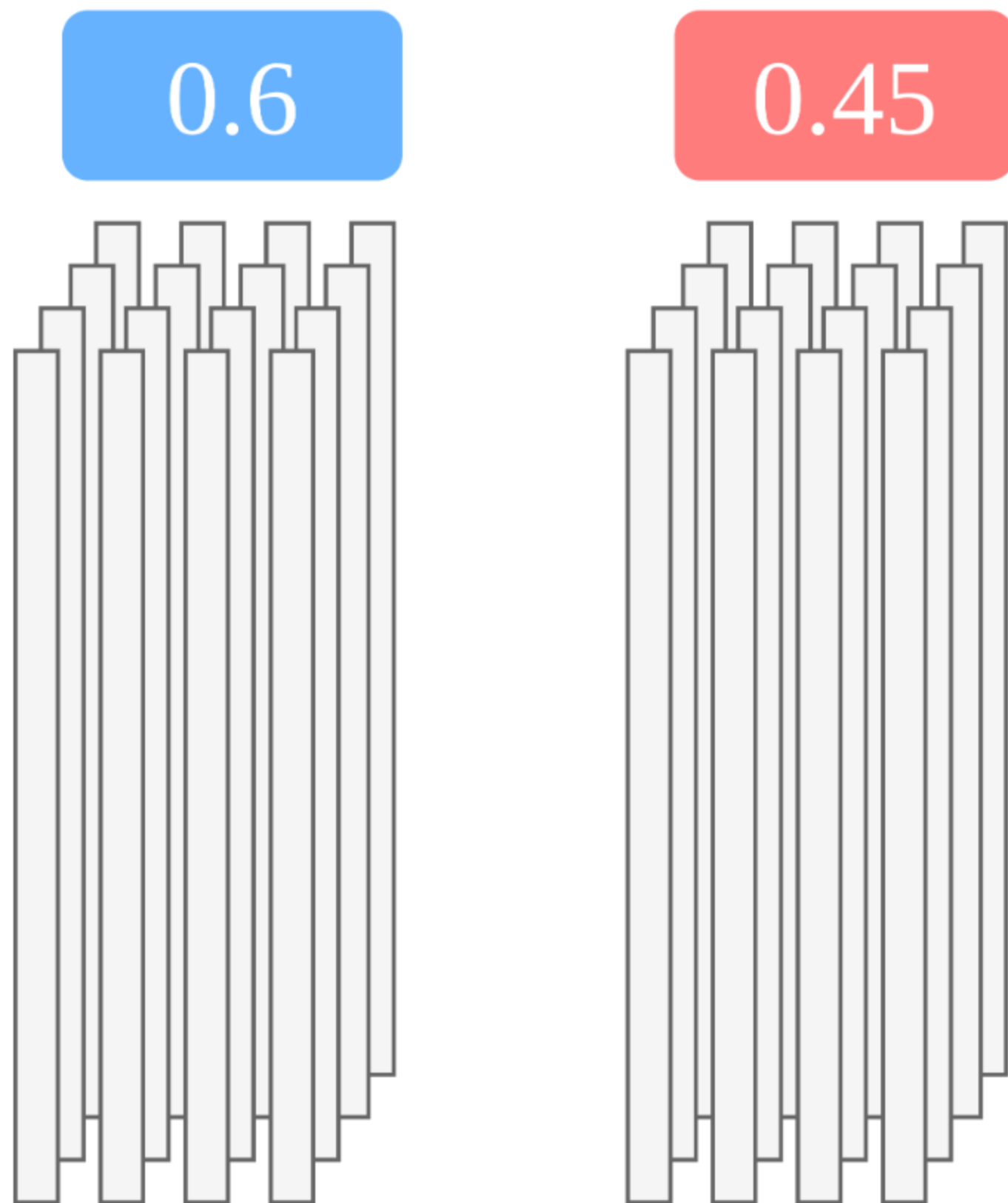
# Learning from Label Proportions (LLP)

- Each user uploads an *unlabeled* 1,034-element vector every day
  - Number of visits to the top 517 sites
  - Number of times referred to the top 517 sites
- Unlabeled vectors are grouped by state
- Each state has a ground-truth label



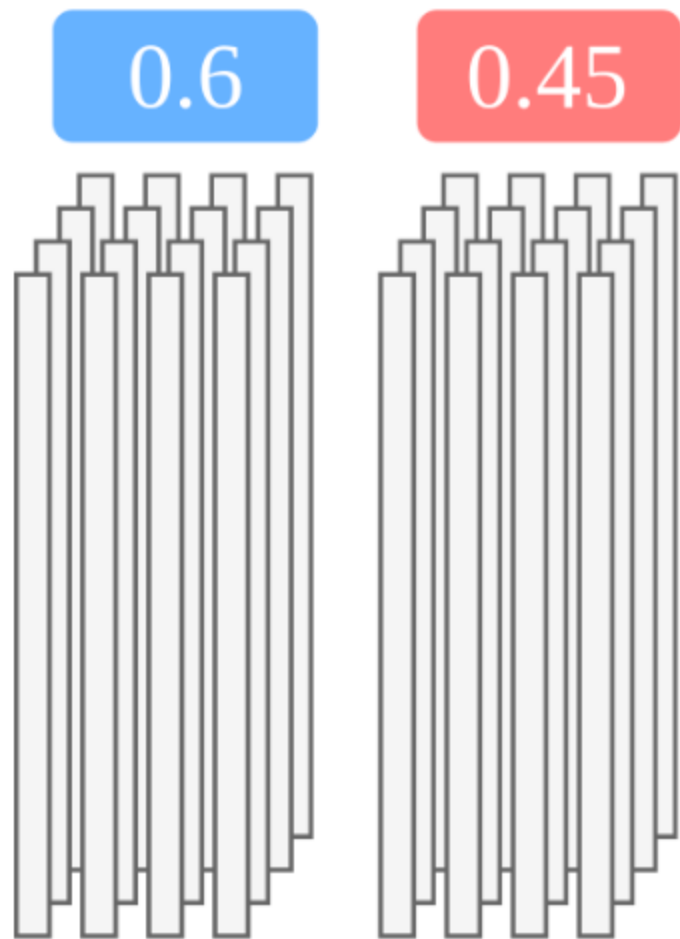
# Learning from Label Proportions (LLP)

- Each user uploads an *unlabeled* 1,034-element vector every day
  - Number of visits to the top 517 sites
  - Number of times referred to the top 517 sites
- Unlabeled vectors are grouped by state
- Each state has a ground-truth label
- Train on aggregate ground truth



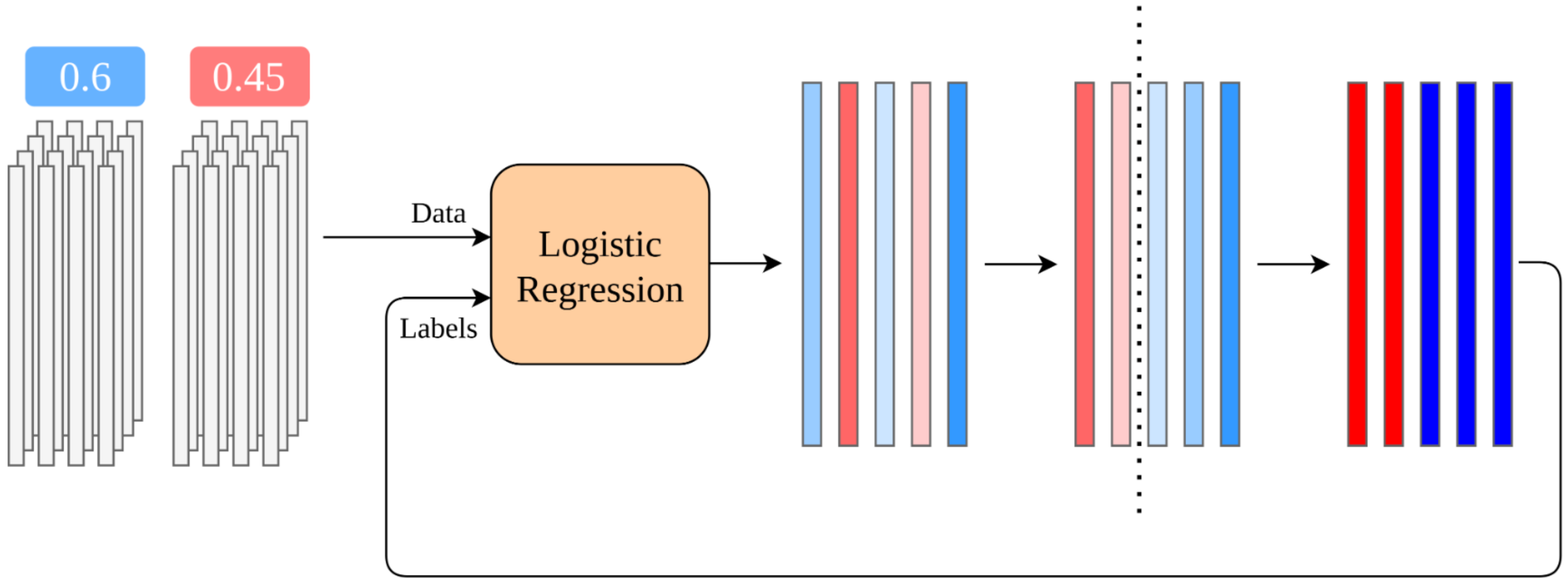
# The Plaintext Algorithm

# The Plaintext Algorithm





# The Plaintext Algorithm



Repeat Until Convergence

# Translating to MPC

```
initialize_labels()

repeat until convergence:
    model = train(data, labels)
    predictions = inference(model, data)

    for each state:
        compute_threshold(truth[state], predictions[state])

    for each user:
        labels[user] = predictions[user] >= thresholds[state]
```

# Translating to MPC

```
initialize_labels()

repeat until convergence:
    model = train(data, labels)
    predictions = inference(model, data)

    for each state:
        compute_threshold(truth[state], predictions[state])

    for each user:
        labels[user] = predictions[user] >= thresholds[state]
```

- Plaintext initialization



# Translating to MPC

```
initialize_labels()

repeat until convergence:
    model = train(data, labels)
    predictions = inference(model, data)

    for each state:
        compute_threshold(truth[state], predictions[state])

    for each user:
        labels[user] = predictions[user] >= thresholds[state]
```

- Plaintext initialization
- Training and inference as black boxes

# Translating to MPC

```
initialize_labels()

repeat until convergence:
    model = train(data, labels)
    predictions = inference(model, data)

    for each state:
        compute_threshold(truth[state], predictions[state])

    for each user:
        labels[user] = predictions[user] >= thresholds[state]
```

- Plaintext initialization
- Training and inference as black boxes
- Computing thresholds requires oblivious sorting

# Translating to MPC

```
initialize_labels()

repeat until convergence:
    model = train(data, labels)
    predictions = inference(model, data)

    for each state:
        compute_threshold(truth[state], predictions[state])

    for each user:
        labels[user] = predictions[user] >= thresholds[state]
```

- Plaintext initialization
- Training and inference as black boxes
- Computing thresholds requires oblivious sorting
- Updating labels requires oblivious comparison

# Translating to MPC

```
initialize_labels()

repeat until convergence:
    model = train(data, labels)
    predictions = inference(model, data)

    for each state:
        compute_threshold(truth[state], predictions[state])

    for each user:
        labels[user] = predictions[user] >= thresholds[state]
```

- Plaintext initialization
- Training and inference as black boxes
- Computing thresholds requires oblivious sorting
- Updating labels requires oblivious comparison

## Convergence Check

# Translating to MPC

```
initialize_labels()

repeat until convergence:
    model = train(data, labels)
    predictions = inference(model, data)

    for each state:
        compute_threshold(truth[state], predictions[state])

    for each user:
        labels[user] = predictions[user] >= thresholds[state]
```

- Plaintext initialization
- Training and inference as black boxes
- Computing thresholds requires oblivious sorting
- Updating labels requires oblivious comparison

## Convergence Check

- Vector oblivious comparison



# Translating to MPC

```
initialize_labels()

repeat until convergence:
    model = train(data, labels)
    predictions = inference(model, data)

    for each state:
        compute_threshold(truth[state], predictions[state])

    for each user:
        labels[user] = predictions[user] >= thresholds[state]
```

- Plaintext initialization
- Training and inference as black boxes
- Computing thresholds requires oblivious sorting
- Updating labels requires oblivious comparison

## Convergence Check

- Vector oblivious comparison
- Sum the comparison results

# Translating to MPC

```
initialize_labels()

repeat until convergence:
    model = train(data, labels)
    predictions = inference(model, data)

    for each state:
        compute_threshold(truth[state], predictions[state])

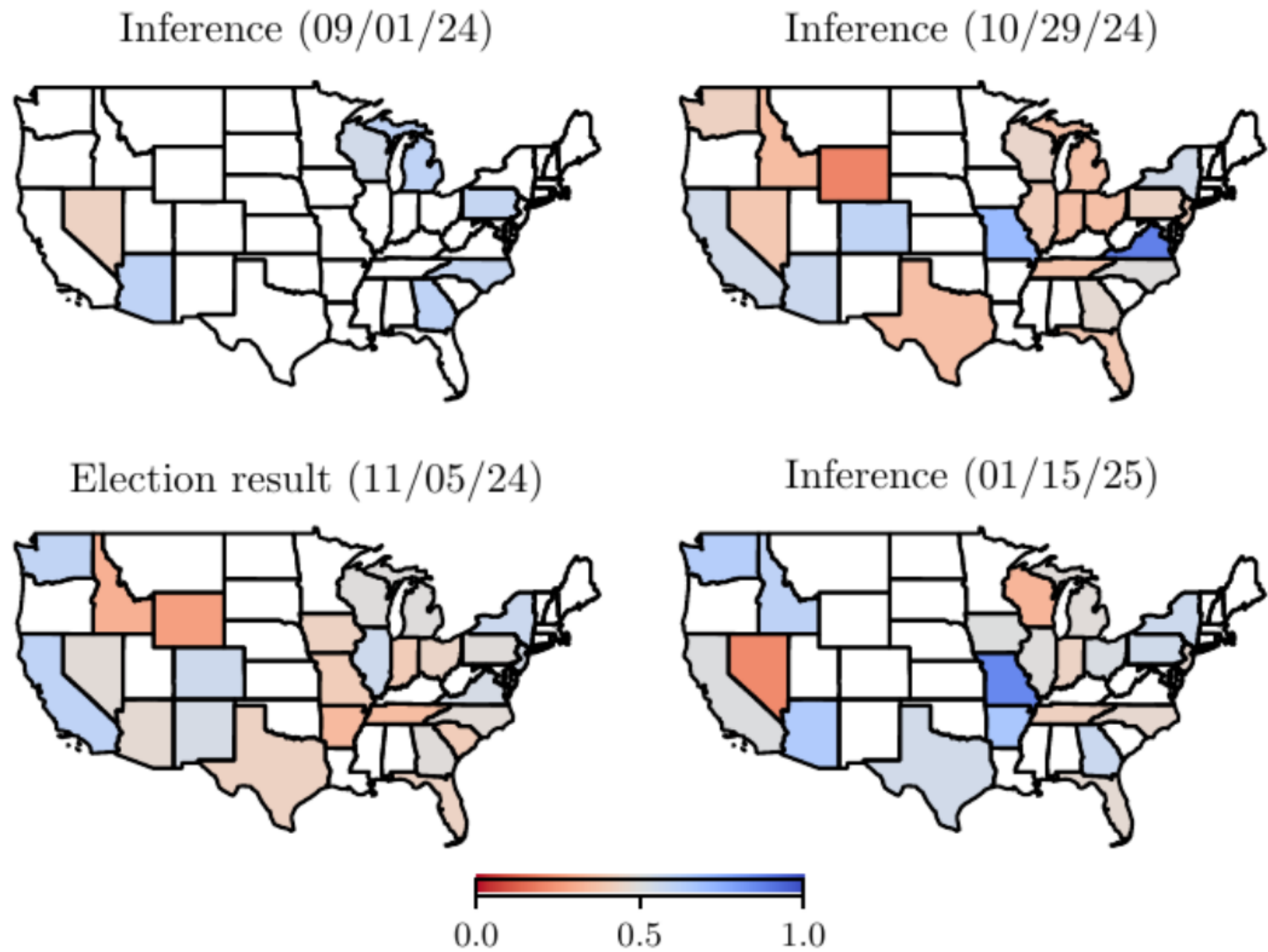
    for each user:
        labels[user] = predictions[user] >= thresholds[state]
```

- Plaintext initialization
- Training and inference as black boxes
- Computing thresholds requires oblivious sorting
- Updating labels requires oblivious comparison

## Convergence Check

- Vector oblivious comparison
- Sum the comparison results
- Compare sum with convergence threshold

# Preliminary Results





# Lessons Learned and Future Directions

# Data Integrity Matters

# Data Integrity Matters

- Initial trouble with dishonest location reporting

# Data Integrity Matters

- Initial trouble with dishonest location reporting
- Validation with IP addresses and geolocation

# Data Integrity Matters

- Initial trouble with dishonest location reporting
- Validation with IP addresses and geolocation
- Results were mixed



# Data Integrity Matters

- Initial trouble with dishonest location reporting
- Validation with IP addresses and geolocation
- Results were mixed
  - Fully verified 15% of users

# Data Integrity Matters

- Initial trouble with dishonest location reporting
- Validation with IP addresses and geolocation
- Results were mixed
  - Fully verified 15% of users
- Digging deeper on the data

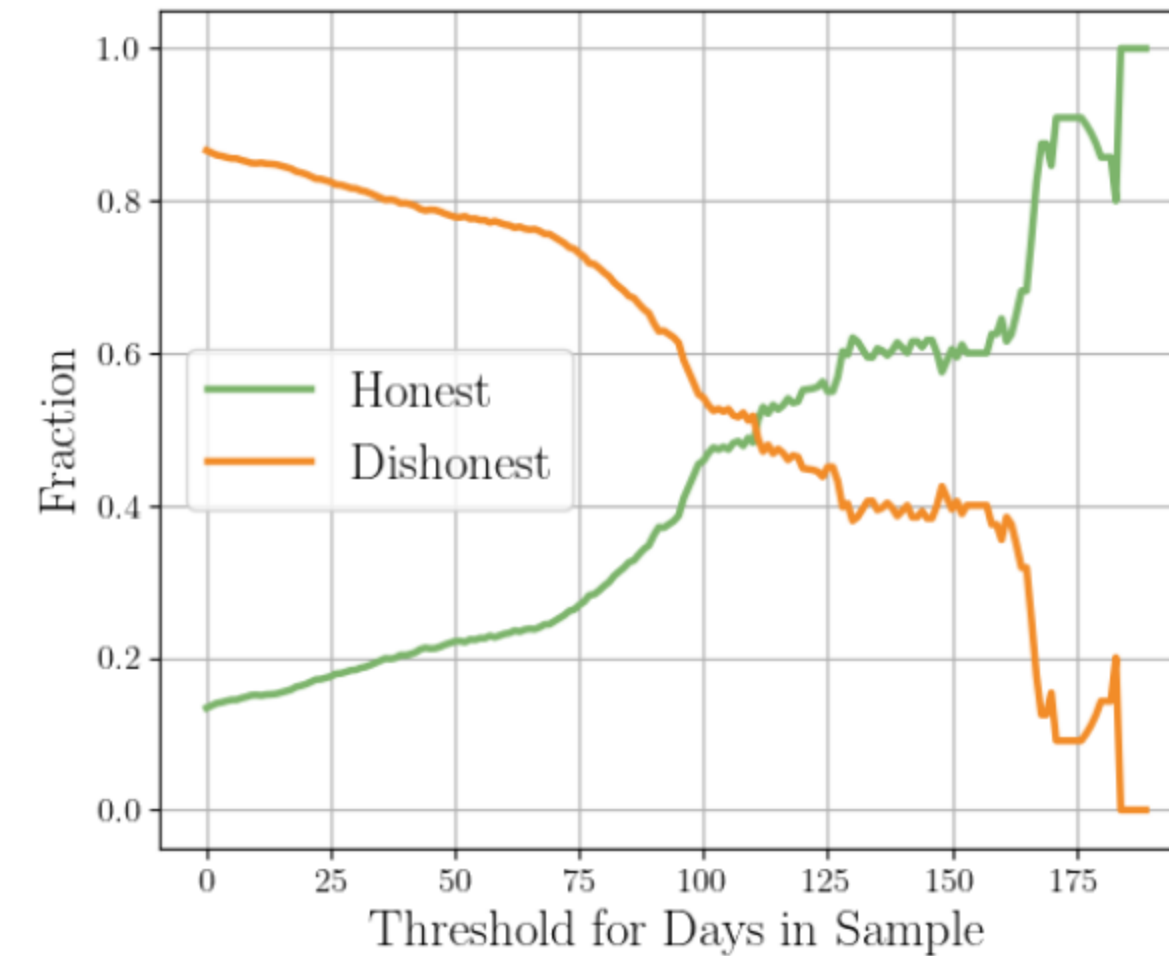
# Data Integrity Matters

- Initial trouble with dishonest location reporting
- Validation with IP addresses and geolocation
- Results were mixed
  - Fully verified 15% of users
- Digging deeper on the data
  - Users in the sample for longer are more honest



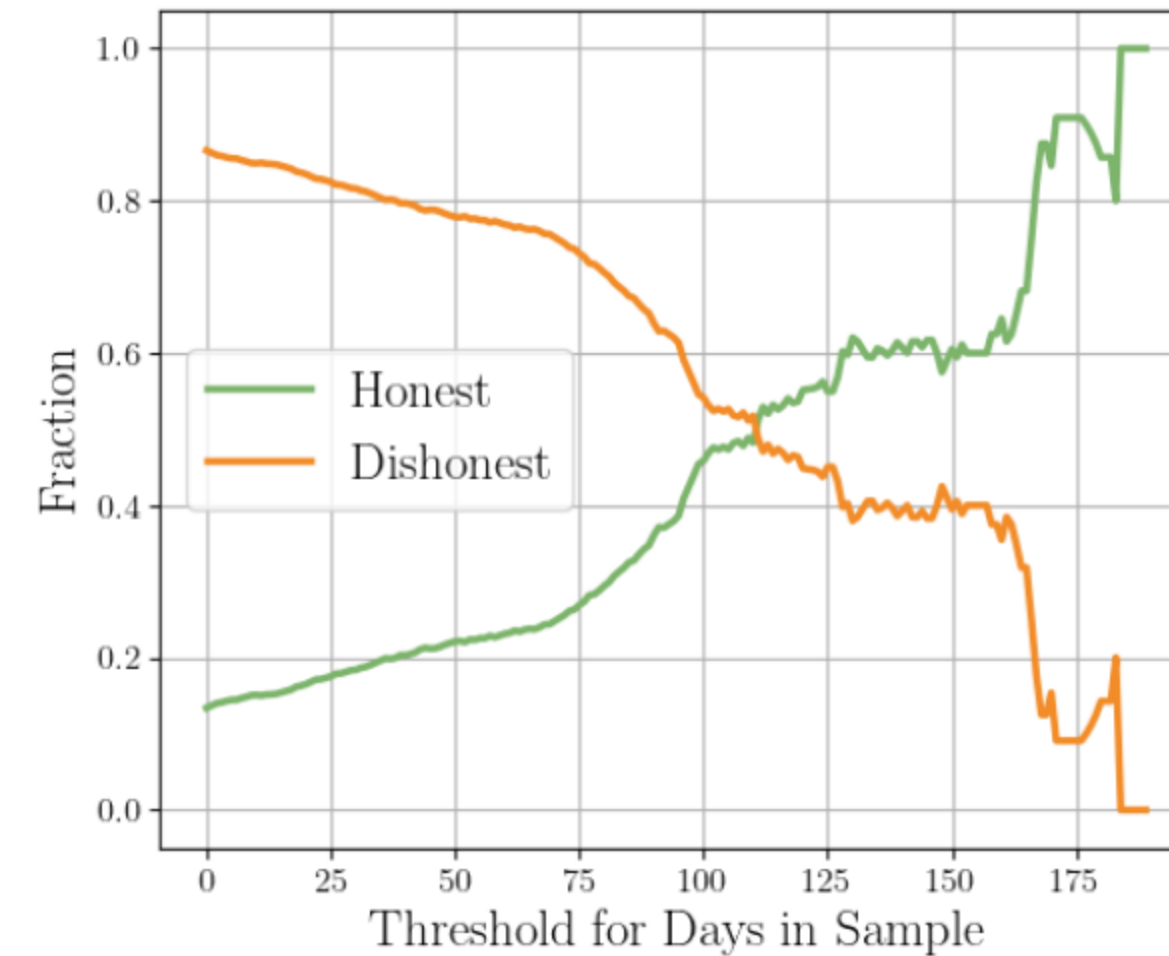
# Data Integrity Matters

- Initial trouble with dishonest location reporting
- Validation with IP addresses and geolocation
- Results were mixed
  - Fully verified 15% of users
- Digging deeper on the data
  - Users in the sample for longer are more honest



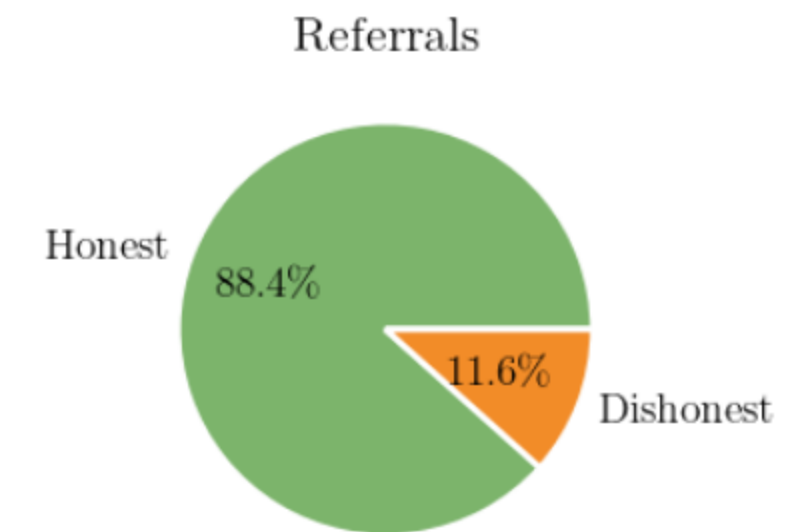
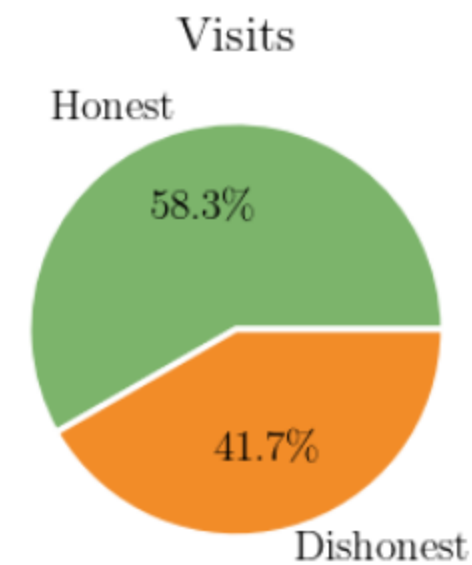
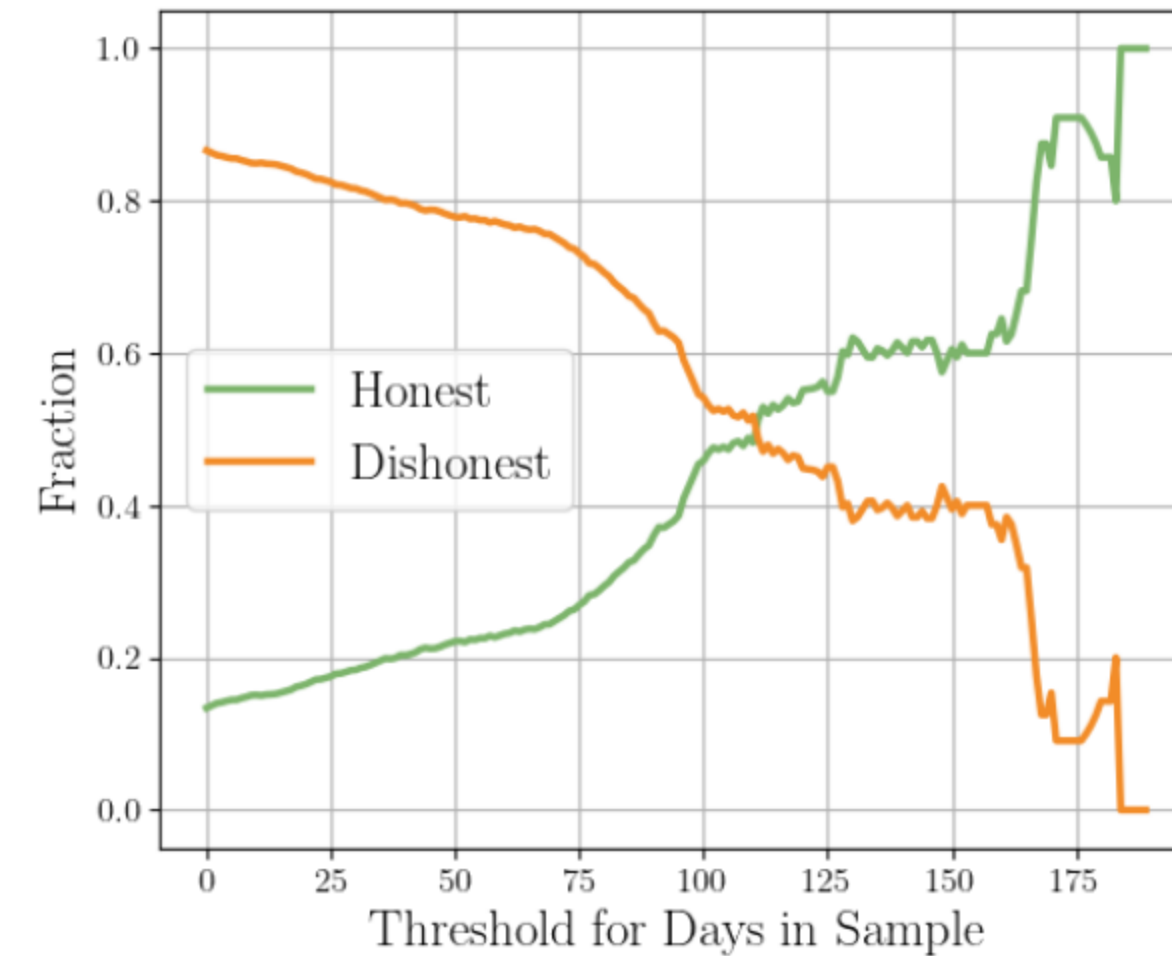
# Data Integrity Matters

- Initial trouble with dishonest location reporting
- Validation with IP addresses and geolocation
- Results were mixed
  - Fully verified 15% of users
- Digging deeper on the data
  - Users in the sample for longer are more honest
  - Honest users contribute much richer data



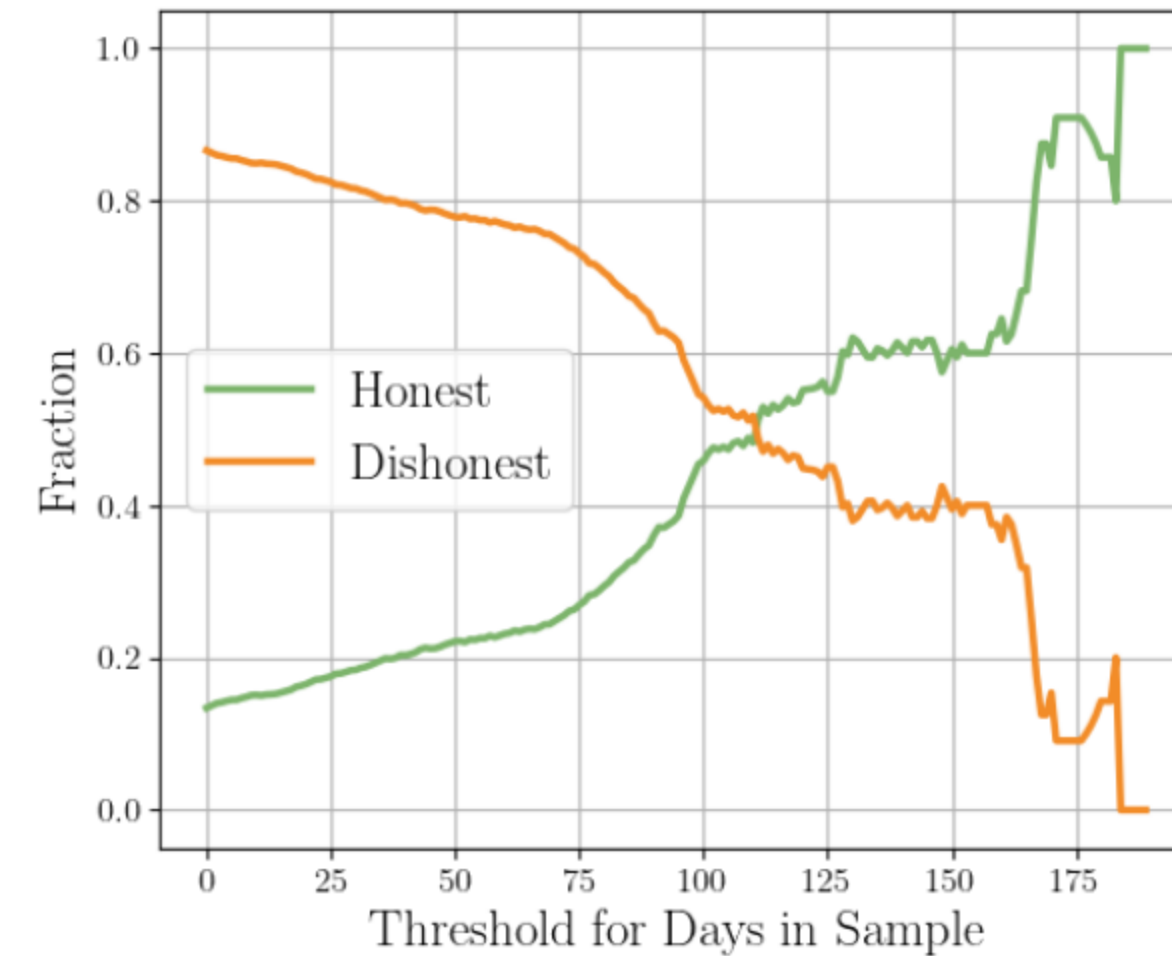
# Data Integrity Matters

- Initial trouble with dishonest location reporting
- Validation with IP addresses and geolocation
- Results were mixed
  - Fully verified 15% of users
- Digging deeper on the data
  - Users in the sample for longer are more honest
  - Honest users contribute much richer data

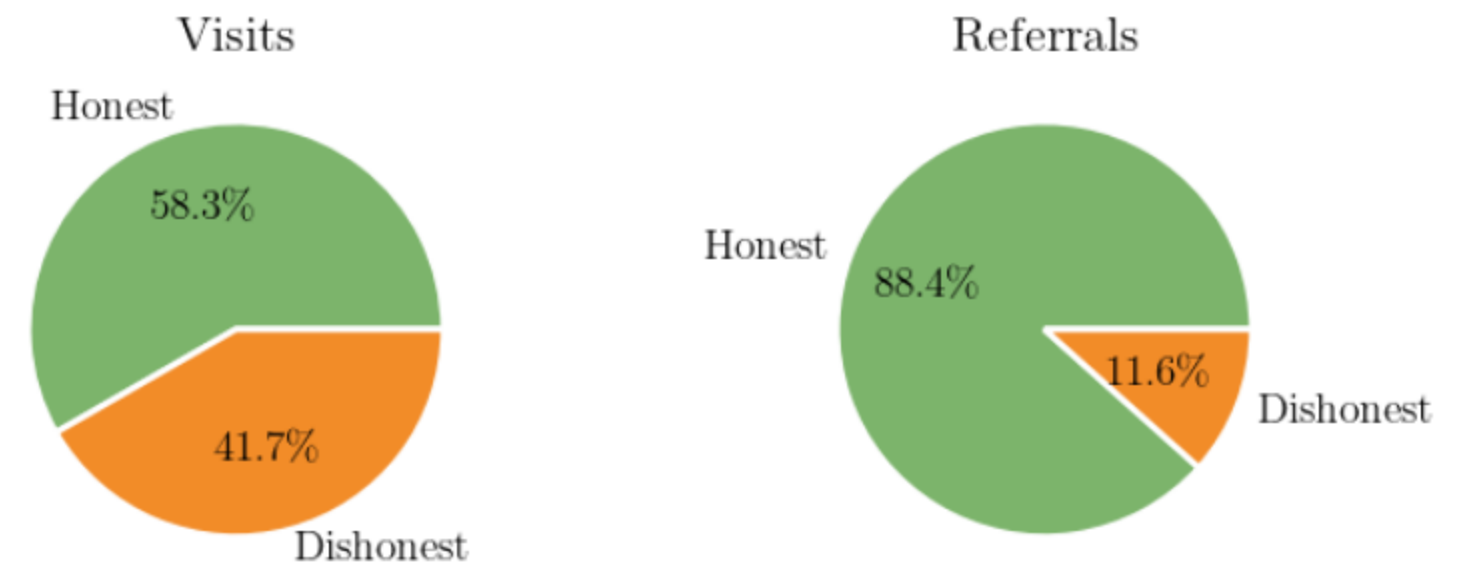


# Data Integrity Matters

- Initial trouble with dishonest location reporting
- Validation with IP addresses and geolocation
- Results were mixed
  - Fully verified 15% of users
- Digging deeper on the data
  - Users in the sample for longer are more honest
  - Honest users contribute much richer data

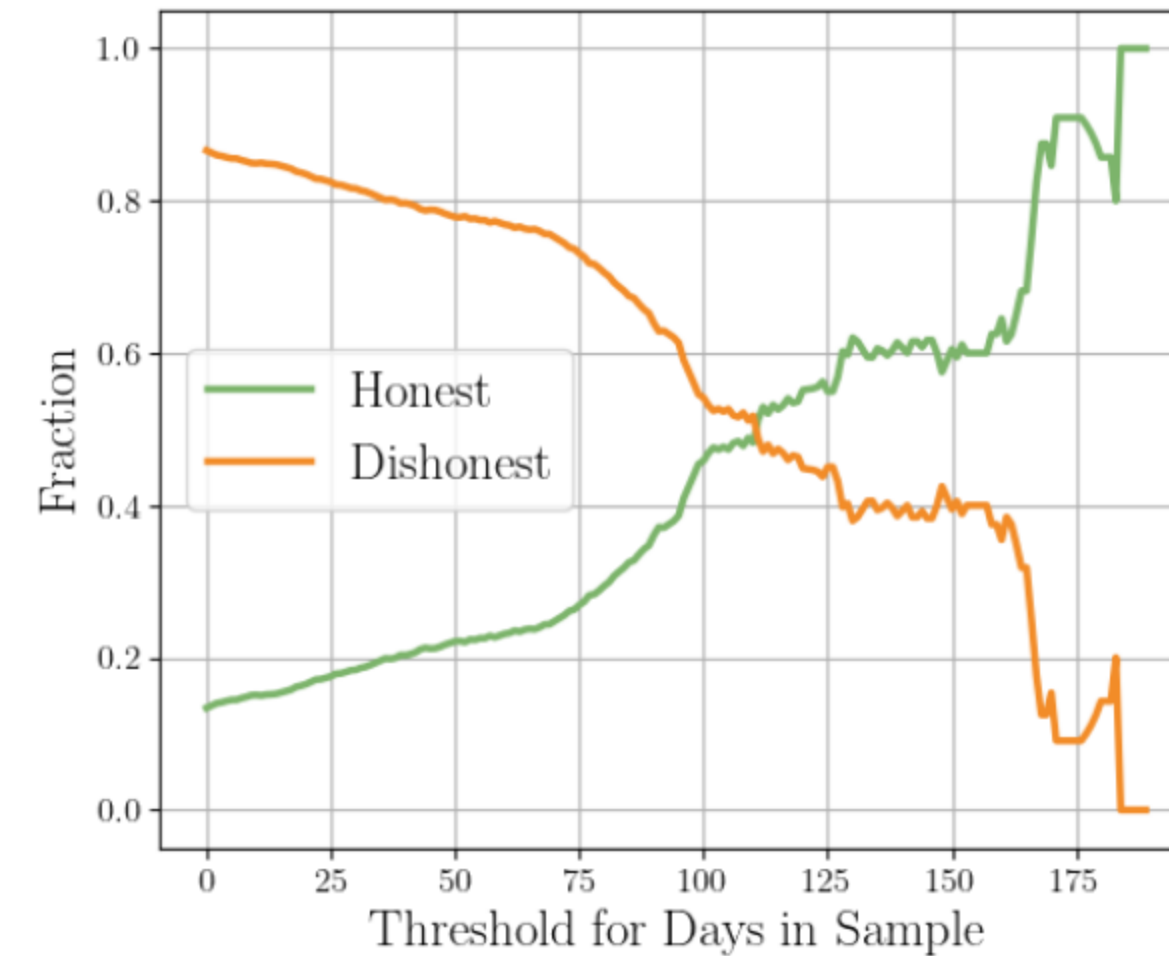


**Lesson:** Validating and enforcing user honesty should be a priority in future deployments.



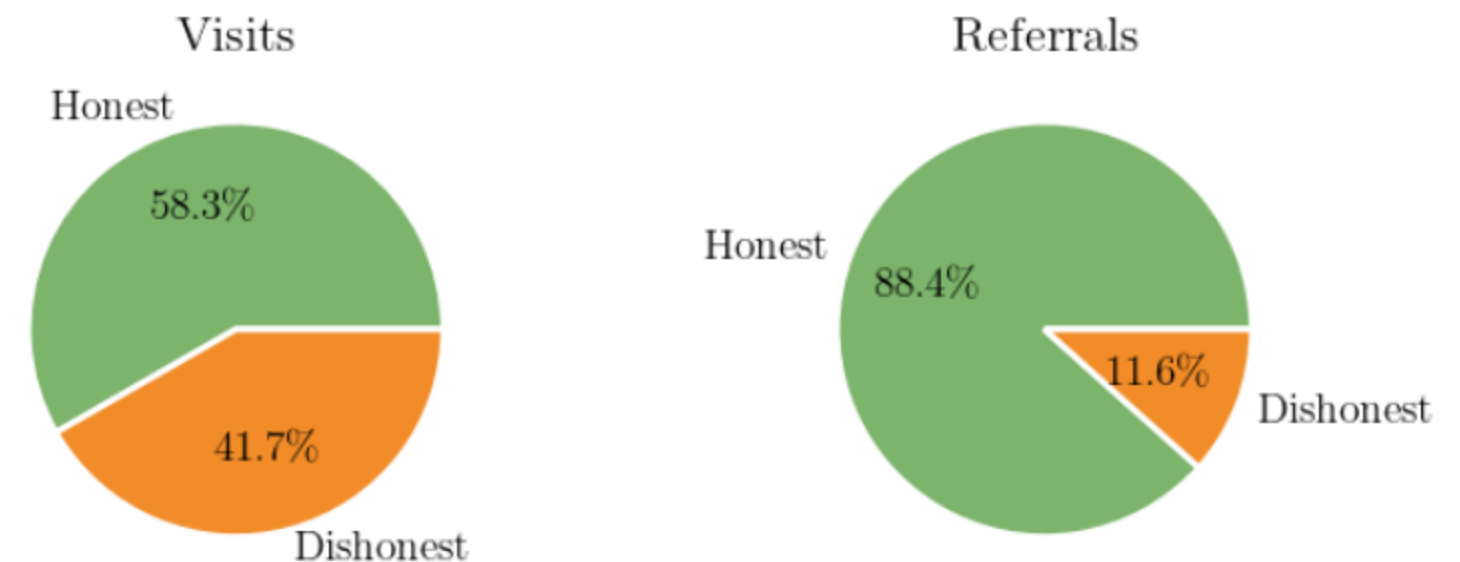
# Data Integrity Matters

- Initial trouble with dishonest location reporting
- Validation with IP addresses and geolocation
- Results were mixed
  - Fully verified 15% of users
- Digging deeper on the data
  - Users in the sample for longer are more honest
  - Honest users contribute much richer data



**Lesson:** Validating and enforcing user honesty should be a priority in future deployments.

**Lesson:** Our data is surprisingly robust to dishonest users.





# Strengthening the Threat Model

# Strengthening the Threat Model

- AWS as a single point of failure

# Strengthening the Threat Model

- AWS as a single point of failure
- Reduce or eliminate trust in the core computation



# Strengthening the Threat Model

- AWS as a single point of failure
- Reduce or eliminate trust in the core computation
- Anonymous payments

# Thank You!

sambux@bu.edu

